

慎思式结构中的环境模型一般是预先知道的,因而对动态环境存在一定的局限性,不适用于未知环境。由于缺乏必要的知识资源,在 Agent 执行时需要向模型提供有关环境的新信息,而这往往难以实现。慎思式 Agent 具有较高的智能,但无法对环境的变化做出快速响应,而且执行效率相对较低。

11.1.6 复合式 Agent

复合式 Agent 综合了反应式 Agent 和慎思式 Agent 的优点,具有较强的灵活性和快速的响应性。复合式 Agent 是在一个 Agent 内组合多种相对独立和并行执行的智能形态,其结构包括感知、动作、反应、建模、规划、通信和决策等模块,如图 11.5 所示。

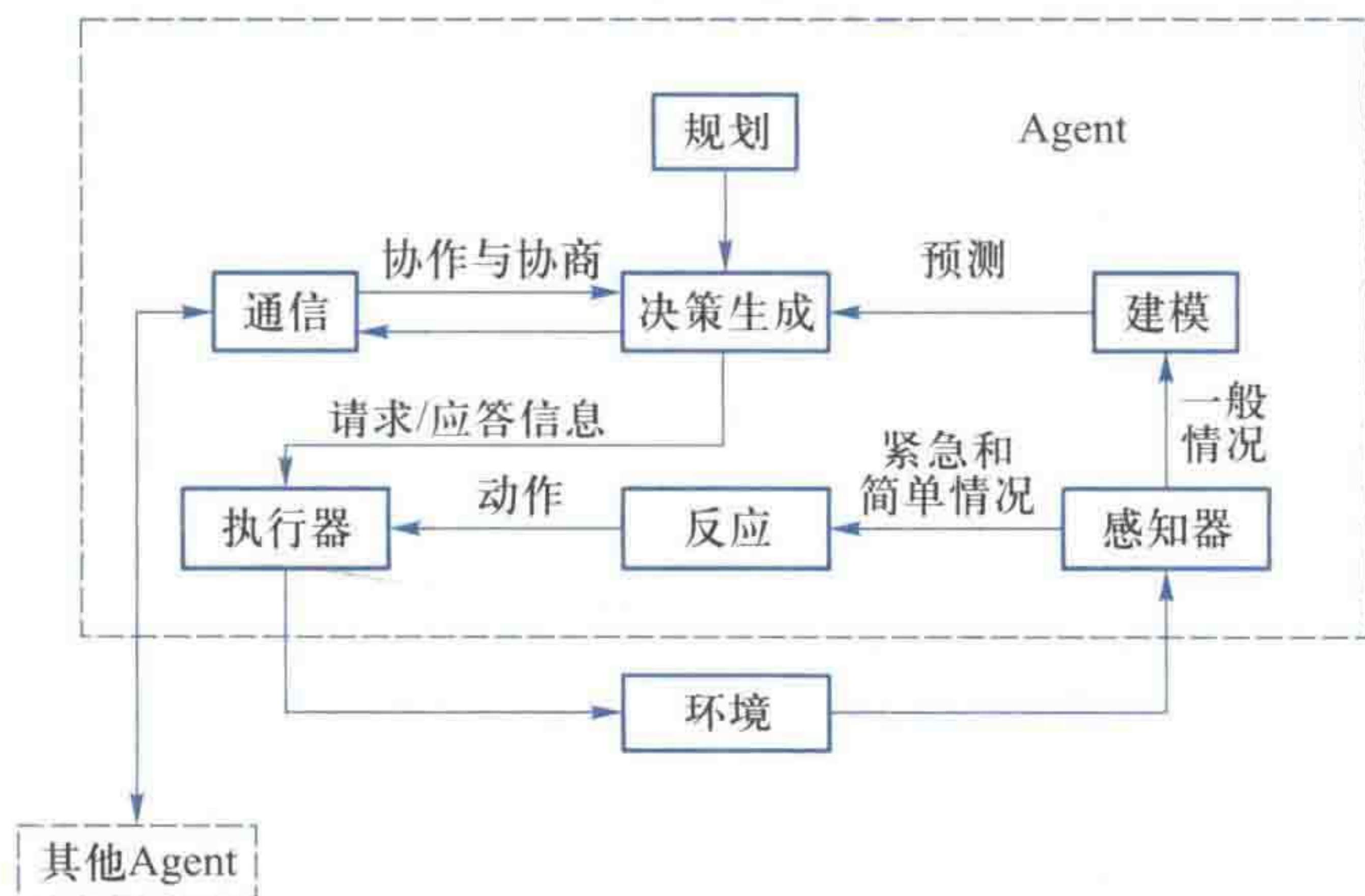


图 11.5 复合式 Agent 的结构示意图

Agent 通过感知模块来感知外部环境信息,再送到不同的处理模块。若感知紧急和简单情况,信息就被送入反映模块,做出决定,并把动作命令送到行动模块,产生相应的动作。

11.1.7 AI Agent 的应用

随着 Agent 技术的逐步成熟,Agent 已经被应用于如下很多领域。

① 电信。该领域主要是利用 Agent 的自主性、协作性、可移动性和自适应性去解决复杂系统和网络管理方面的任务,包括负载均衡、故障预测、问题分析和信息综合等。

② 兴趣匹配。Agent 更多应用于商业网站向用户提供建议。例如,MIT 多媒体实验室的研究人员在这个领域做了很多的工作,相应的研究成果也曾被用于亚马逊书店和一些销售唱片和影碟的网站中。

③ 用户助理。用 Agent 协助用户更好地完成特定的任务。所使用的 Agent 都体现在用户界面层次,为用户完成某些特定的任务提供相应的信息和建议。

④ 组织结构。由多个 Agent 构造一个类似于人类组织的系统,不同的 Agent 代表着系统内的不同的角色,通过这些 Agent 之间的通信和协作来完成具体的任务。目前主要应用于电子商

务。例如,一个典型的多 Agent 供应链系统中就包含购买者 Agent、供应商 Agent 和中介 Agent 等多种 Agent。

⑤ 智能信息检索。Agent 可以通过利用相关知识检索一些特定信息。用于信息服务的智能 Agent 能够告诉用户所需要的资源在哪里,根据网上资源回答用户特定主题的问题,按照用户指定的条件过滤筛选信息,以及帮助用户整理下载的信息,还能够从大量的公共原始数据中筛选和提炼出有价值的信息,向有关用户发布。

⑥ 决策支持系统。一般决策支持系统通常都需要大量综合性的信息,以及对信息经过深度加工的结果和知识。在这类系统中 Agent 能够监控系统的一些关键信息,在系统可能出现问题的時候,提醒相应的操作员,并在数据挖掘技术和决策支持模型的协助下,为复杂的决策提供有效的支持。

⑦ 移动计算。Agent 的自适应性将使网络服务更有效地适应于各种类型的数据通信模式和移动终端,而且 Agent 的离线计算能力还能为移动应用提供自然有效且稳定的离线计算模式,即使在移动用户断开与网络的连接之后,Agent 仍然能够继续完成尚未完成的任务,并在移动用户再次连上网络之后把结果反馈给用户。除此之外,Agent 还能够为移动用户提供友好个性化的界面,从而为用户提供个性化的服务。

⑧ 远程教育。在远程教育系统中引入 Agent,作为虚拟教师、虚拟学习伙伴、虚拟实验设备、虚拟图书管理员等,实现虚拟的教学、练习和实验环节等。在单机系统中,可以采用 Agent 技术设计人性化的角色,实施对学习者的导航的模式,增加教学内容的趣味性,改善计算机辅助教学效果。

⑨ 数字娱乐。在网络数字娱乐系统中引入 Agent,可以增强娱乐效果。例如,在个性化的节目中插入点播服务,在游戏、动画中进行更加人性化的角色设计。

⑩ 智能家居系统。智能家居设备通过自主学习用户习惯,自动调整温度、照明和安保系统,提升居住舒适度和安全性。

⑪ 智能医疗系统。智能医疗系统在推荐治疗方案时,需要遵循医疗伦理,确保患者的健康和平安是首要考虑因素。

⑫ 智能客服。智能客服系统通过分析用户反馈,逐步形成以客户满意度为核心的价值观,从而提升服务质量和用户体验。

⑬ 自动驾驶汽车。自动驾驶汽车通过自主感知环境、决策行驶路径和控制车辆,实现了高度的自主性。这些汽车能够在复杂的交通环境中独立行驶,处理各种突发情况。在设计自动驾驶系统时,开发者需要考虑伦理决策,例如在不可避免的碰撞情况下,如何选择保护乘客或行人的价值观。

⑭ 智能机器人。在智能制造领域,工业机器人能够根据生产需求,自主规划生产流程,优化资源使用,执行组装、检验和包装等多种任务,提高生产效率。服务机器人可以在酒店、餐厅等场所执行接待、送餐、清洁等多项任务,展示了其多功能性和适应能力。

目前,大量的自主人工智能产品已经广泛使用,已经渗透到更多行业,尤其是那些需要高

度自动化与智能化的领域。智能体不仅在工业、医疗、交通等传统领域发挥作用,还将在农业、环境保护等新兴领域展现出巨大潜力。例如,哥伦比亚大学研制的面向科学研究的 GPT Researcher 和具备联网搜索功能的 AutoGPT 等,这些产品以人工智能大模型为核心,AI 智能体通过互联网实时获取最新信息,基于大语言模型进行思考,取得更好的效果。

LLM 也有缺陷。例如 ChatGPT、Bard 等 LLM 的产品形态都是对话机器人,专精于对话、绘画等特定场景的问题,普遍存在能力固化。而且,有限的算力也限制了 LLM 的记忆力,从而难以满足更多用户的使用。OpenAI CEO 山姆·奥特曼就曾表示,由于 GPU 短缺导致算力不足,无法扩大 ChatGPT 的对话框列表,直接影响到了回答用户问题时处理的信息量,限制了 ChatGPT 的“记忆力”被限制。

11.2 多智能体系统的概念与结构

对于现实中复杂的大规模问题,只靠单个的智能体往往无法描述和解决。因此,一个应用系统中往往包含多个智能体,这些智能体不仅具备自身的问题求解能力和行为目标,而且能够相互协作,达到共同的整体目标,这样的系统称为多智能体系统(multi-agent system, MAS)。

11.2.1 多智能体系统的特点

① MAS 中每个智能体具有独立性和自主性,能够解决给定的子问题,自主地推理和规划并选择恰当的策略,并以特定的方式影响周围的环境。

② MAS 支持分布式应用,具有良好的模块性,易于扩展,设计简单灵活,克服了建造大规模知识库时的知识管理和扩展的困难。

③ MAS 按面向对象的方法构造多层次、多元化的智能体,降低了系统的复杂性,也降低了各个智能体问题求解的复杂性。

④ MAS 是一个协调式的系统,各个智能体之间相互协调合作可以解决大规模的复杂问题;MAS 也是一个集成系统,它采用信息集成技术,将各子系统信息集成起来。

⑤ 在 MAS 中,智能体之间相互通信,彼此协调,并行地求解问题,提高了问题求解效率。

⑥ 同一个 MAS 中各个智能体可以是异构的。因此,多智能体技术为各种复杂系统提供了一种统一的模型。

⑦ 在 MAS 中,不同领域的专家系统、同一领域不同的专家系统可以协作求解单一专家系统难以解决的问题。多智能体技术打破了当前知识工程领域中仅使用一个专家系统的限制。

11.2.2 多智能体系统的基本类型

针对不同的应用环境,从不同角度提出了多种类型的多智能体模型,包括 BDI 模型、协商模型、协作规划模型和自协调模型等。不同于数学的机械证明,人的推理是一种实证推理,有 3 个关键要素:信念(belief)、期望(desire)和意图(intention),把这样的实证推理逻辑形式化为

BDI 逻辑,而把实现 BDI 逻辑的智能体结构称为 BDI 结构。

1. BDI 模型

BDI 模型是一个概念和逻辑上的理论模型,是研究 Agent 理性和推理机制的基础。在把 BDI 模型扩展到 MAS 的研究时,提出了联合意图、社会承诺、合理行为等 Agent 行为的形式化定义。联合意图为 Agent 建立复杂动态环境下的协作框架,对共同目标和共同承诺进行描述。当所有 Agent 都同意这个目标时,就一起承诺去实现该目标。联合承诺可以用来描述合作推理和协商,社会承诺给出社会承诺机制。

2. 协商模型

Agent 的协作行为一般通过协商而产生。虽然各个 Agent 的行动目标是使自身效用最大化,然而在完成全局目标时,就需要各 Agent 在全局上建立一致的目标。对资源缺乏的 Agent 动态环境,任务分解、任务分配、任务监督和任务评价就是一种必要的协商策略。合同网协议就是协商模型的典型代表,主要解决任务分配、资源冲突和知识冲突等问题。

3. 协作规划模型

多智能体系统的规划模型主要用于制定其协调一致的问题规划。每个 Agent 都具有自己的求解目标,考虑其他 Agent 的行动约束,并进行独立规划。网络结点上的部分规则可以用通信方式来协调所有结点,达到所有 Agent 都接受的全局规划。部分全局规划允许各 Agent 动态合作。Agent 的相互作用以通信规划和目标的形式抽象表达,以通信原语描述规划目标,相互告知自己的期望行为,利用规划信息调节自身的局部规划,达到共同目标。

4. 自协调模型

自协调模型是建立在开放和动态环境下的 MAS 模型,能够随环境变化自适应地调整行为。该模型的动态性表现在系统组织结构的分解重组和 MAS 内部的自主协调等方面。

11.2.3 多智能体系统的体系结构

MAS 的体系结构影响系统的异步性、一致性、自主性和自适应性的程度,决定信息的存储方式、共享方式和通信方式。体系结构中必须有共同的通信协议或传递机制。对于不同复杂程度的应用,应选择不同能力的体系结构。

下面介绍几种常见的 MAS 的体系结构。

1. 网络结构

网络结构中的 Agent 之间都是直接通信的,通信和状态知识都是固定的。网络结构的 Agent 系统中的每个 Agent 必须知道消息应该在什么时候发送到什么地方,系统中有哪些 Agent 是可以合作的,都具备什么样的能力等。但是,将通信和控制功能都嵌入到每个 Agent 内部,就是要求系统中的每个 Agent 都拥有有关其他 Agent 的大量信息和知识,而在开放的分布式系统中这往往是做不到的。另外,当系统中 Agent 的数目越来越多时,这种一对一的直接交互将导致低效率。

2. 联盟结构

联盟结构的工作方式是:若干相距较近的 Agent 通过一个叫作协助者的 Agent 进行交互,而远程 Agent 之间的交互和消息发送是由局部 Agent 群体的协助者 Agent 协作完成的。这些协助者 Agent 可以实现各种各样的消息发送协议。当一个 Agent 需要某种服务时,它就向它所在的局部群体的协助者 Agent 发送一个请求,该协助者 Agent 将以广播方式发送该请求,或者将该请求与其他 Agent 所声明的能力进行匹配,将此信息发送给对它感兴趣的 Agent。这种结构中的 Agent 不需要知道其他 Agent 的详细信息,因此有较大的灵活性。协助 Agent 能够实现一些高层系统服务,如黄页、直接通信、问题分解和监控等。

3. 黑板结构

黑板结构和联盟系统有相似之处,不同的地方在于黑板结构中的局部 Agent 把信息存放在可存取的黑板上,实现局部数据共享。在一个局部 Agent 群体中,控制外壳 Agent(类似于联盟结构中的协助者)负责信息交互,而网络控制者 Agent 负责局部的 Agent 群体之间的远程信息交互。黑板结构的不足之处在于:局部数据共享要求一定范围的 Agent 群体中的 Agent 拥有统一的数据结构或知识表示,这就限制了系统中 Agent 设计和建造的灵活性。因此,开放的分布式系统不宜采用黑板结构。

11.3 多智能体系统的通信

同一环境中的 Agent 之间、不同环境的 Agent 之间都可以进行通信。一个 Agent 被授权后就可以通过调用另一个 Agent 的方法向它发送消息。

Agent 进行通信就是改变信息载体,将载体发送到接收 Agent 的可观察环境中。采用这种通信观点可以拓展通信的形式,而不仅限于语言通信。

两个 Agent 之间的通信过程如下:

- ① 发送方将自己的思想翻译成通信所用语言的格式;
- ② 发送方将声音、文字和图像等的语言格式加载到通信传播媒体中;
- ③ 传播载体到达接收方;
- ④ 接收方读取载体中的语言代码;
- ⑤ 接收方在思维空间中将语言代码按其格式翻译为思想,从而熟悉发送方的意识状态。

11.3.1 智能体通信的类型

通常 Agent 采用两种方式进行通信:一是分享一个共同内部表示语言的 Agent,它们无须任何外部语言就能通信;二是无须做出内部语言假设的 Agent,它们共享英语子集作为通信语言。

1. 使用 Tell 和 Ask 通信

这种通信形式的 Agent 分享共同内部表示语言,并通过界面 Tell 和 Ask 直接访问共享的知识库。

Agent A 可以使用 $\text{Tell}(\text{KB}_B, "P")$ 通信把提议 P 传到 Agent B, 就如 A 会使用 $\text{Tell}(\text{KB}_B, "P")$ 把 P 加到自己的知识库; 类似地, Agent A 可以使用 $\text{Ask}(\text{KB}_B, "Q")$ 查出 B 是否知道 Q 。称为灵感通信 (telepathic communication)。如图 11.6 所示, 两个共享内部语言的 Agent 使用 Tell 和 Ask 界面并借助于知识库相互直接通信, 其中每个 Agent 除了具有感知和行为端口之外, 还有一个到知识库的输入和输出端口。

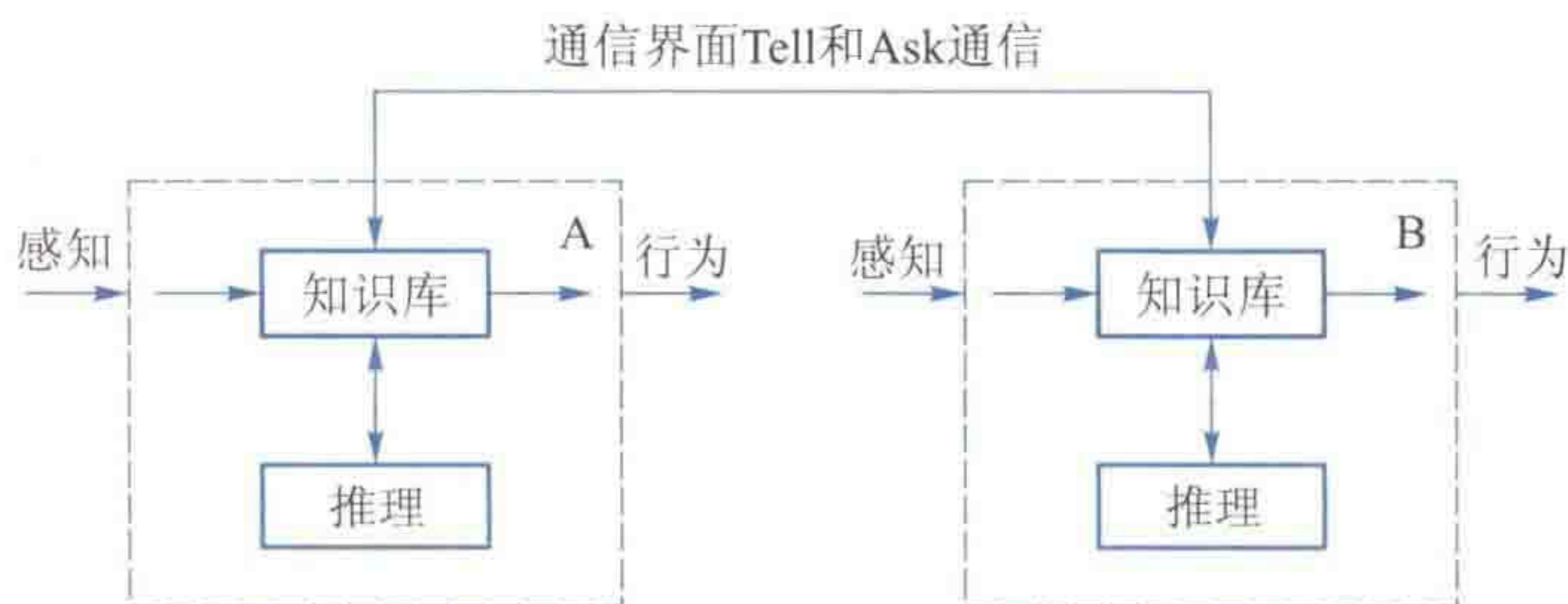


图 11.6 两个 Agent 通过 Tell 和 Ask 通信

2. 使用形式语言通信

大多数 Agent 的通信是通过语言而不是通过直接访问知识库而实现的。两个 Agent 使用语言通信如图 11.7 所示。有的 Agent 可以执行表示语言的行为, 而其他 Agent 可以感知这些语言。外部通信语言可以与内部表示语言不同, 并且这些 Agent 中的每一个都可以有不同的内部语言。只要每个 Agent 能够可靠地从外部语言映射到自己的内部语言, 他们就无须同意任何内部符号。

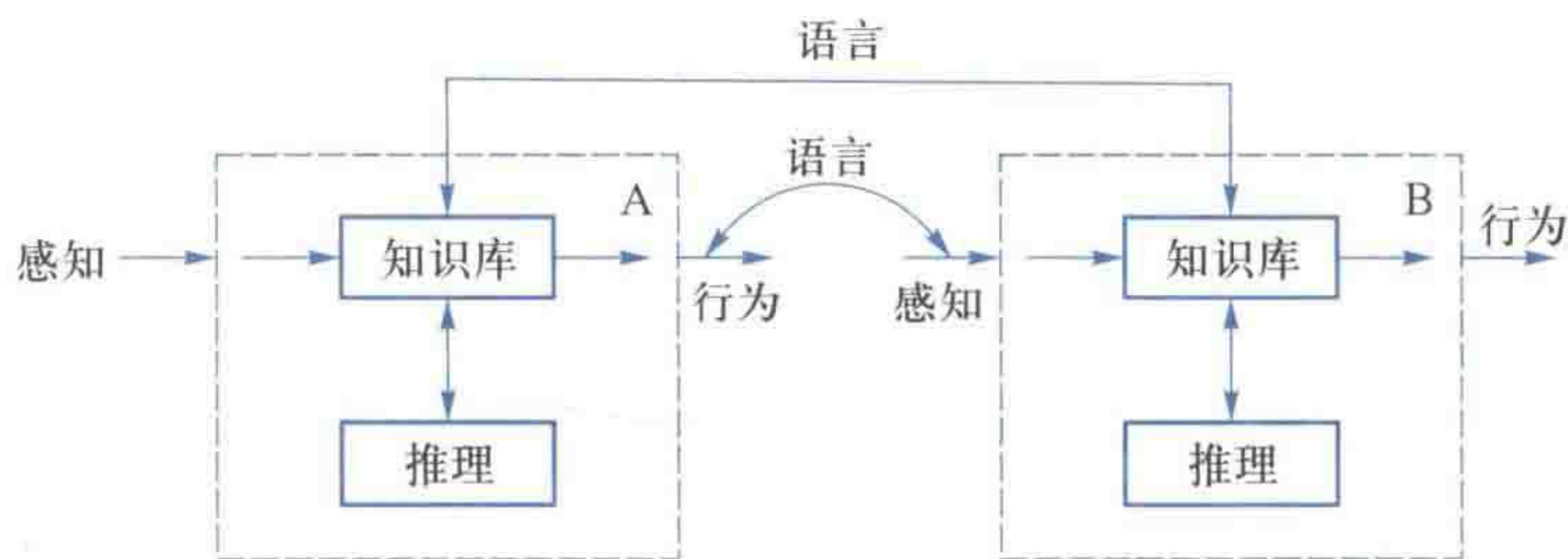


图 11.7 两个 Agent 使用语言通信

对于外部通信语言生成和分析的问题, 主要由自然语言处理方法来设计这两个过程的算法。但是语言通信最难的部分是前面提到的问题: 协调不同的 Agent 的知识库之间的差别。

11.3.2 Agent 通信的方式

Agent 之间的通信是多智能体系统的基本功能。通信可分为黑板系统和消息/对话系统两种方式。

1. 黑板系统

黑板系统主要支持分布式问题求解。在多智能体系统中,黑板提供公共工作区,参与求解的 Agent 可以“看”到黑板上的问题、数据、求解记录,并将自己对问题的求解结果“写”到黑板上,以供其他 Agent 进行参考、求解。在这一模型中,Agent 之间不能进行直接的通信,信息的交换都是通过黑板这一共享媒介完成的,每个 Agent 独立完成各自求解的问题。

黑板系统可用于任务共享系统和结果共享系统。如果黑板中的 Agent 很多,那么黑板中的数据就会剧增。各个 Agent 在访问黑板时,需要从大量信息中搜索并提取感兴趣的信息。为进行优化处理,黑板应为各 Agent 提供不同的区域。

黑板系统的模型有以下三个主要组成部分。

- ① 知识源,即 Agent,是作为求解问题的独立单元,具有不同的专门知识,独立完成特定的任务。
- ② 黑板,即公共工作区,为知识源提供信息和数据,同时,供知识源进行修改。
- ③ 监控机制。根据黑板当前的问题求解状态,以及各知识源的不同求解能力,对其进行监控,使之能实时发现黑板变化,及时进行问题求解。

2. 消息/对话系统

消息/对话系统方式是实现灵活和复杂的协调策略的基础。各 Agent 使用规定的协议相互交换信息,用于建立通信和协调机制。为了支持协作策略,通信协议必须明确规定通信过程、消息格式和选择通信的语言。另外,Agent 之间的通信是知识级的通信,参与通信的 Agent 必须知道通信语言的含义。

有如下两种方式来实现 Agent 间的消息传递。

- ① 在直接通信方式中,每个 Agent 必须知道消息应该在什么时候发送到什么地方,系统中有哪些 Agent 是可以合作的,这些 Agent 各具备什么样的能力等。这要求系统中的每个 Agent 都拥有关于其他 Agent 的大量信息。
- ② 通过中介的通信在基于中介的消息传送中,若干相距较近的 Agent 通过通信服务器来进行交互和消息发送,而远程 Agent 之间的交互是由局部群体中的通信服务器协作完成的。

11.3.3 智能体通信语言

智能体之间的交互和协调是通过智能体之间的通信来完成的。当前智能体在完成了对其他智能体的建模之后,如果要对其他智能体进行控制和协调,比如改变其他智能体的目标、知识或者是对环境的置信,应该通过通信动作来完成。

多智能体系统一个最显著的特征是各个智能体在满足自身利益的同时,通过相互之间的协商和合作,共同完成单一智能体无法完成的任务。智能体要进行协商和合作,相互之间的通信是必不可少的环节。智能体是具有一定智能性和自主性的软件实体,因此,智能体之间通信所传递的不仅仅是字符流或二进制数据流,而且在知识层上进行表达、理解和交流。

虽然在计算机界已经出现了一些互通的标准和方法,如 CORBA、OLE、ActiveX 等,但这些

标准都是从动态、分布环境下运行应用程序困难的角度出发,主要解决数据结构交换和跨平台的远程调用问题。智能体之间的交互并不仅仅局限于数据和方法,因而这些标准都不能彻底解决智能体之间的通信问题。

由美国 KSE (Knowledge-Sharing Effort) 机构研究开发的智能体通信语言 (agent communication language, ACL) 是目前智能体通信研究中最引人注目的。KSE 主要研究开发支持不同系统之间知识共享的基础技术。该机构包括了三个不同的工作组,分别负责中间语言 (interlingua)、可共享重用知识 (shared reusable knowledge) 和外部接口 (external interface) 的研究工作。

目前国际上最流行的智能体通信语言有两种:一种称为知识交换格式语言;另一种称为知识查询操纵处理语言。

1. 知识交换格式语言

知识交换格式语言 (knowledge interchange format, KIF) 主要是基于谓词逻辑,可以作为描述专家系统、数据库、多智能体的知识表示工具。KIF 负责将一种语言翻译成另一种语言,或者为两种异构智能体的知识表达提供语义共享。可共享重用知识则是一个词汇表,它可以使可共享知识库的内容更容易被理解,同时也为特定的领域提供开发工具和方法。外部接口主要设计软件智能体运行时能够共享知识和信息的通信高层语言 (knowledge query and manipulation language, KQML), 即知识查询操纵语言。KQML 是自主的异步智能体之间共享知识和实现协作问题求解的通信语言。它既是一种消息格式,也是支持实时智能体之间知识共享的消息处理协议,其目的在于实现基于知识的异构系统之间实现互操作和集成。上述的 KIF、词汇表和 KQML 三个部分正好构成了智能体通信语言的主体。

在实际应用中,软件实体之间的互操作和知识共享是应用程序作为软件智能体的重要基础,甚至是必不可少的先决条件。图 11.8 显示了互操作软件智能体的抽象模型。

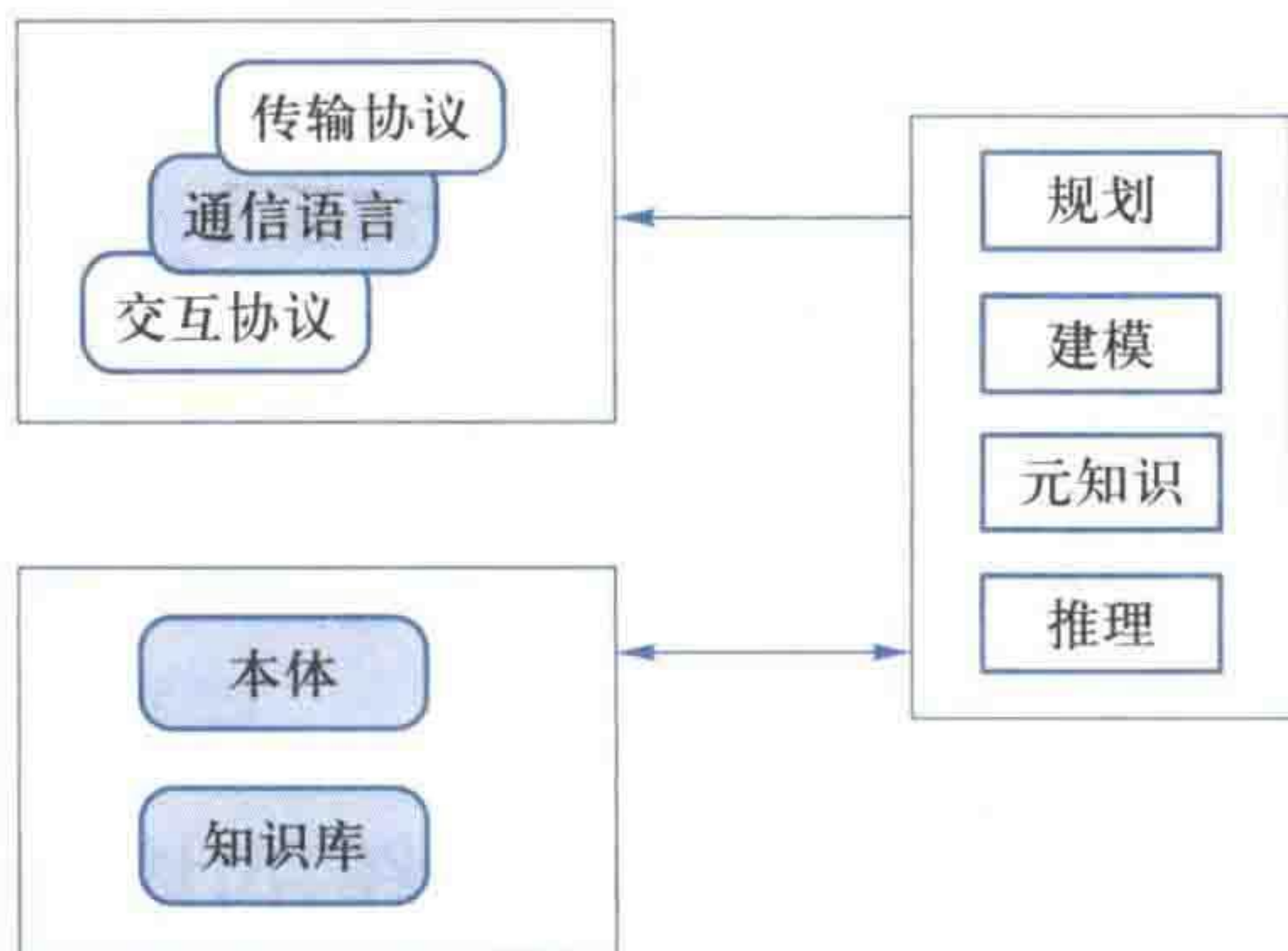


图 11.8 互操作软件智能体的抽象模型

2. 知识查询操纵语言

KQML 为多智能体通信定义了一套消息表达机制和消息传递格式,从而为多智能体通信

提供了一套建立连接识别和交换消息的协议,构建了一种标准通用框架。

KQML 是一种层次结构语言,可以分为 3 层:内容层、通信层和消息层。内容层使用应用程序本身的表达语言来传送消息的实际内容。KQML 可以携带用任何表示语言编写的表达式,包括那些 ASCII 字符串和基于二进制符号表达的内容。通信层主要负责对消息的某些特性进行编码,这些特性描述了底层通信参数,如发送者和接收者的标识符。

消息层是整个 KQML 语言的核心。将一条消息从一个应用程序传送到另一个应用程序时,消息层完成对所传送信息的封装。消息层的一个最基本功能是识别传输消息发送时所使用的协议,并且给消息发送者提供一个附加在内容上的述行语或原语(performative)。除此之外,由于通信内容对参与交互的智能体来说是不透明的,因此消息层还必须包含描述内容的语言、假设的本体和一些内容的类型描述等特征。这些特征使得 KQML 语言在内容不可知的情况下实现对消息的分析、路由和正确的传送。

KQML 语言中有多种在进程之间进行信息交换的协议。最简单的情况,一个智能体充当客户端并向另一个充当服务器的智能体发出一个请求,然后等待服务器方的答复,如图 11.9 中智能体 A 和 B。服务器的应答可以是单一的,也可以包含一组回答。一般服务器方的应答并不是完整的回答,而是提供给客户端一个句柄,客户端可以一次请求一个答复元素,如图 11.9 中的智能体 A 和 C。一个简单的例子就是客户端向关系型数据库请求一系列的实例。图 11.9 中智能体 A 和 D 之间的情况稍微有点不同,当客户端在服务器上注册了一个服务输出后,服务器就会不定期地向客户端发送异步应答,而客户端也不知道每一个应答的到达时间。这样做的好处在于客户端可以处理其他任务而不必专门等待服务器的应答信息。

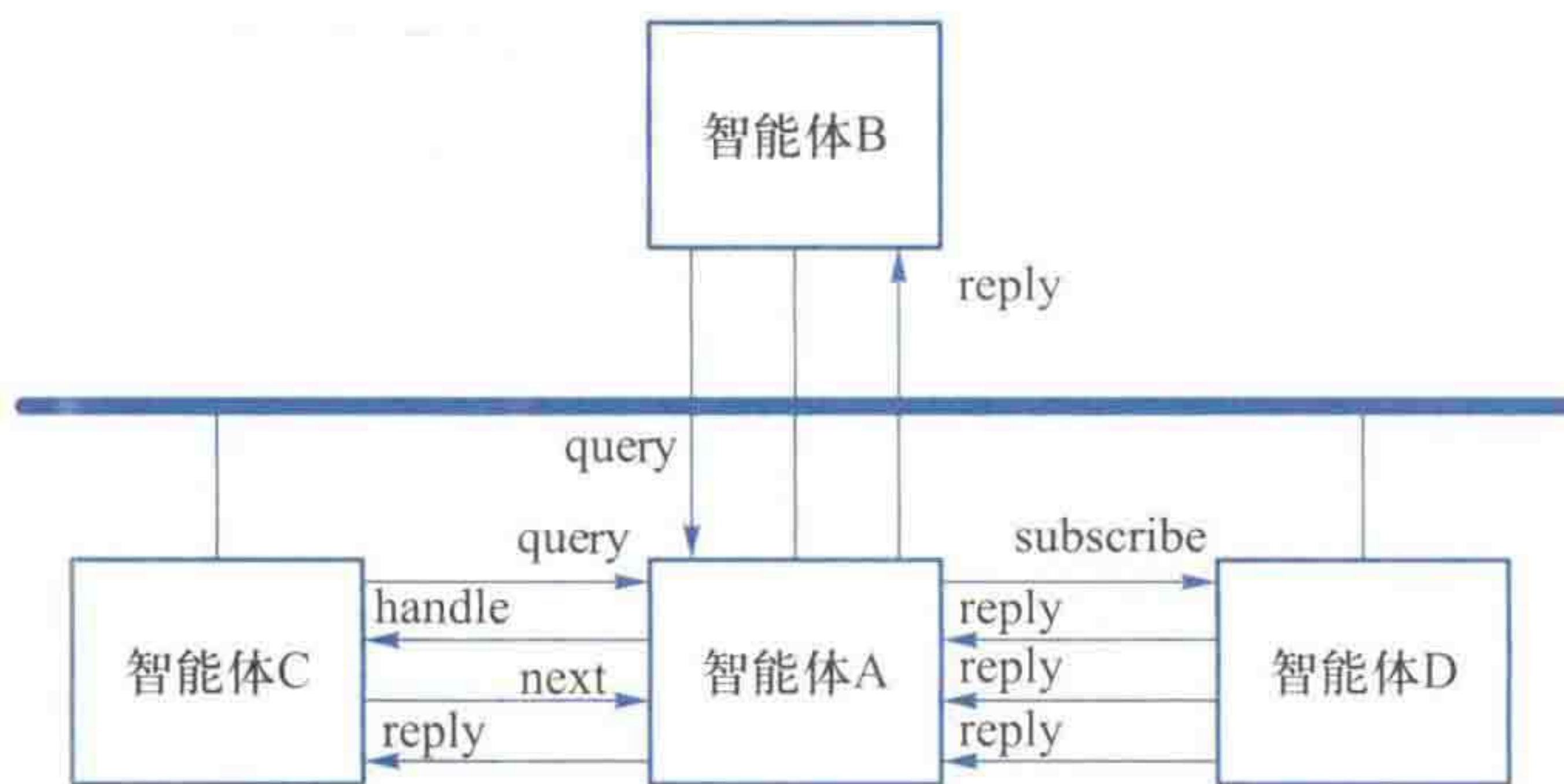


图 11.9 KQML 支持的几种通信协议

11.4 多智能体系统的协调

协调(coordination)和协作(cooperation)是 MAS 研究的核心问题。协调是指一组 Agent 完成一些集体活动时相互作用的性质。协作是非对抗的 Agent 之间保持行为协调的一个特例。Agent 群体协作行为的协调程度、系统的全局连贯性以及多 Agent 组织是否具有自适应性,是

关系到整个系统性能和开放性的核心,同时也是一个系统智能水平的重要体现。

MAS 中的协调是指多个 Agent 为了一致和谐的方式工作而进行交互的过程。进行协调是希望避免 Agent 之间的死锁或活锁。死锁指多个 Agent 无法进行各自的下一步动作,活锁指多个 Agent 不断工作却无任何进展。

当前主要有以下 4 种协调方法。

(1) 基于集中规划的协调

如果 MAS 中至少有一个 Agent 具备其他 Agent 的知识、能力和环境资源知识,那么该 Agent 可作为主控 Agent 对该 MAS 的目标进行分解,对任务进行规划,并指示或建议其他 Agent 执行相关任务。这种基于集中规划的协调方法特别适用于环境和任务相对固定、动态行为集可预计和需要集中监控的情况。

(2) 基于协商的协调

系统中没有作为规划的主控 Agent,而通过协商来实现任务的分配。协商是 Agent 间交换信息、讨论和达成共识的方式。

(3) 基于对策论的协调

该协调包括有通信的协调和无通信的协调两类。无通信协调是在没有通信的情况下,Agent 根据对方及自身效益模型,按照对策论选择适当行为。在这种协调方式中,Agent 至多也只能达到协调的平衡解。在有通信协调中则可得到协作解。

(4) 基于社会规划的协调

这一类是以每个 Agent 都必须遵循的社会规则、过滤策略、标准和惯例为基础的协调方法。这些规则对于 Agent 的行为加以限制,过滤某些有冲突的意图和行为,保证其他 Agent 必需的行为方式,从而确保本 Agent 行为的可能性,以实现整个 Agent 系统的社会行为的协调。

11.5 多智能体系统的协作

建立多智能体系统的目的在于解决单一智能体难以处理的复杂问题,即任务在时间或空间上的复杂性超过了单个智能体的能力,仅仅依靠单个智能体行为来实现是不可能、不经济、不完整的,因此协作行为就成为多智能体系统必不可少的行为。在多智能体系统中,智能体之间的协调和合作是研究的核心问题之一,也是衡量系统智能水平的重要指标。

11.5.1 多智能体的协作类型

根据智能体之间目标的关系以及协作的程度,可将协作分为如下 5 种类型。

① 完全协作型。系统中的智能体都围绕一个共同的全局目标,各个智能体没有自己的局部目标,所有智能体全力以赴地协作。

② 协作型。系统中的智能体具有一个共同的全局目标,同时各个智能体还有与全局目标一致的局部目标。

③ 自私型。系统中不存在共同的全局目标,各智能体都为自己的局部目标工作,而且目标之间可能存在冲突。

④ 完全自私型。系统中不存在共同的目标,各智能体都为自己的局部目标工作,并且不考虑任何协作行为。

⑤ 协作与自私共存型。系统中既存在一些共同的全局目标,某些智能体也可能还具有与全局目标无直接联系的局部目标。

多智能体系统的协作过程一般分为如下六个阶段。

- ① 产生协作需求,即确定协作目标。
- ② 协作规划,求解合理的协作结构。
- ③ 寻求协作伙伴。
- ④ 选择协作方案,即根据协作竞争者反推最佳的协作方案。
- ⑤ 按协作或交互协议进行协作以实现所确定的目标。
- ⑥ 结果评估,即判断协作的效果并为以后的协作提供可供参考的经验和教训。

下面介绍几种协作方法。

11.5.2 合同网协作方法

在所有的协作方法中,合同网(contract net)是最著名且应用最广泛的一种协作方法。它是 Smith 于 1980 年提出的。基本思想来自人们在商务过程中用于管理商品和服务的合同机制。在合同网方法中,所有智能体分为两种角色:管理者(manager)和工作者(worker)。其中,管理者智能体的职责包括:

- ① 对每一待求解任务建立任务通知书(task announcement),将任务通知书发送给有关的工作者智能体,向工作者发出竞标信息;
- ② 接受并评估来自工作者的投标值(bid);
- ③ 根据事先确定的任务竞标要求,从投标中选择最合适的工作者,并与之签订工作合同(contract);
- ④ 在工作者履行合同的同时,对工作者进行监督和管理。当出现紧急情况的时候,采取适当的应急措施,保证任务的顺利完成,并对结果进行综合和分析。

工作者智能体的职责包括:

- ① 接收相关的任务通知书;
- ② 评价任务书,根据自身的情况设定任务投标的标值;
- ③ 向管理者发送投标信息,其中投标信息包含所投的任务、投标的工作者、投标的标值;
- ④ 若投标获得成功,则和管理者签订合同,并根据合同要求执行合同所分配的任务;
- ⑤ 当所分配的任务完成时,通知管理者,并且报告求解的结果。

在合同网协作方法中,不需要预先规定智能体的角色,智能体在协作过程中的角色可以变化:任何智能体通过发布任务通知书而成为管理者;任何智能体通过应答任务通知书而成为工

作者。这一灵活的特性使得任务能够被层次地分解、分配:系统中每一待求解的任务,由承担该项任务的智能体负责完成。当该智能体无法独立完成该任务时,它就将任务进行分解,为每一子任务发送任务通知书,该智能体从而由工作者变为管理者,履行管理者的职责;然后,从返回的投标中选定“最合适”的工作者智能体,和该工作者智能体签订工作合同,将子任务分配给这一智能体。按合同执行子任务的工作者智能体若不能独立完成任务,就需要再次扮演管理者的角色,将子任务再分解,并按照合同网的方式进行分配。如此进行下去,直到所有任务都能独立完成。从以上步骤可以明显看出,合同网系统的任务分解分配过程形成一个动态确定的树结构。

合同网方法还具有性能平滑衰减的优点:如果一个工作者智能体不能提供一个满意的解或者出现意外不能继续求解时,管理者智能体可以为任务寻找另外一个工作者智能体。这一点对实际生产显得尤为重要,特别是当加工资源(机器)发生故障时,管理者智能体会为分配给该工作者智能体的任务重新寻找合适的新的加工资源,保证任务能够顺利完成,从而提高系统的可靠性和鲁棒性。

11.5.3 黑板模型协作方法

黑板的概念最早是由 Newell 提出的。黑板模型的基本思想为:多个人类专家或智能体专家协同求解一个问题,黑板是一个共享的问题求解工作空间,多个专家都能“看到”黑板。当问题和初始数据记录到黑板上,求解开始。所有专家通过“看”黑板寻找利用其专家经验知识求解问题的机会。当一个专家发现黑板上的信息足以支持他进一步求解问题时,他就将求解结果记录在黑板上。新增加的信息有可能使其他专家继续求解。重复这一过程直到整个问题彻底求解,获得最终结果。

黑板模型由以下三个基本模块构成。

① 知识源。应用领域根据求解问题专门知识的不同划分成若干相互独立的专家,这些专家称为知识源(即智能体)。每一知识源独立完成一个特定领域的任务。

② 黑板。共享的问题求解工作空间。一般是以层次结构的方式组织,主要存放知识源所需要的信息和求解过程中的解状态数据,如初始数据、部分解、替换解、最终解等,有时也存放控制数据。在问题求解过程中,知识源不断地修改黑板。知识源之间的通信和交互智能通过黑板进行。

③ 监控机制。根据黑板上的问题求解状态和各知识源的求解技能,依据某种控制策略,动态地选择和激活合适的知识源,使知识源能实时地响应黑板的变化。

11.5.4 市场机制协作方法

上述两种智能体协作机制要求智能体之间直接通信,一般适合数量较少的智能体之间的协作求解,对于异构环境下的大量智能体之间的协作,市场机制则是较好的解决方案。

20 世纪 90 年代末期,电子商务和虚拟企业等复杂系统的发展,迫切需要研究异构智能体

如何自主地实现协作的问题。这里的异构智能体可能是由不同的人,在不同的时间,利用不同的软件工具、技术,实现从属于不同组织的智能体。这些应用需求促进新的协作技术的出现。

市场机制的基本思想是针对分布式资源分配的待定问题,建立相应的计算经济,使智能体之间通过最少的直接通信来协调多个智能体之间的活动。在该方法中,对智能体关心的所有事务(如技能、资源等)都给予标价。系统中只存在两种类型的智能体:生产者和消费者。生产者能够提供服务,即将某一商品转换为另一商品;消费者能够进行商品交换。智能体以各种价格对商品进行投标,但所有的商品交换都以当前市场价格进行,每一智能体通过投标以便获得最大的利益和效用。

在开放的市场环境中,智能体应该可以自由地选择自己的贸易策略,其行为不一定合乎常规。市场机制假定智能体所给予的偏好是与智能体获得行动结果的知识一致,因此智能体推理行为就是最大化它自身的偏好。

11.6 多智能体系统的协商

协商(negotiation)是MAS实现协调、协作、冲突消解和矛盾处理的关键环节。有关MAS协商的关键技术可以概括为协商协议、协商策略和协商处理这三方面的内容。

1. 协商协议

协商协议用于处理协商过程中协商方之间的交互和作用,是交易双方交互的规则,决定何时何方采用何种行为,是规范交易协商行为的基础。

它主要研究的内容是Agent通信语言的定义、表示、处理和语义解释。协商协议的最简单形式如下:

一条协商通信消息:(<协商原语>,<消息内容>)

其中,协商原语即消息类型,它的定义通常基于言语行为理论;消息内容除包含消息的发送者、接收者、消息号、发送时间等固定信息外,还包括与协商应用的具体领域相关的信息描述。

协商协议的形式化表示通常有三种方法:巴科斯范式表示、有限自动机表示和语义表示。巴科斯范式表示具有简洁明了的特点,是最常用的表示方法。采用纯语义表示的协商工作不多,研究者更多的是给出非形式化的语义解释。常用的协商协议有:根据协商对象的数量分为一对一、一对多、多对多的协议;根据协商的顺序分为轮流出价、同时出价协商协议;根据协商议题的数量分为单属性和多属性协商等。

2. 协商策略

协商策略是Agent选择协商协议和通信消息的策略。一般来说,协商策略分为提议评估策略和提议生成策略两部分。提议评估策略用来对收到的提议进行评估,判断是否接受对方给出的提议;提议生成策略用来生成反提议。策略对于协商的效率起着至关重要的作用,根据不同的应用领域可以选择不同的协商策略。

协商策略基本上可以分为五类:单方让步策略、竞争型策略、协作型策略、破坏协商策略和

拖延协商策略。单方让步策略只是在协商陷入僵局或协商不再有意义时才起作用,后两类策略显然不利于推进协商进程,所以,只有竞争型和协作型策略才是有意义的。

竞争型策略一般是指协商参与者坚持自己的立场,在协商过程中表现出竞争行为,使协商结果向有利于自身利益方向发展的协商对策。合同网协调模型、劳资协商、基于对策论的协商过程等都属于此类。协作型策略则是指协商各方都从整体利益出发,在协商过程中互相合作,他们采取的协商对策有利于寻找互相能接受的协商结果。

不论是竞争型策略,还是协作型策略,Agent 应动态地、智能地选择适宜的协商策略,从而在系统运行的不同时刻表现出不同的竞争或协作行为。

策略选择的通用方法是:依据影响协商的多方面因素,给出适宜的策略选择函数。策略选择函数可能包括效用函数、比较或匹配函数、兴趣或爱好函数等几种。策略选择函数的设计除了要考虑影响协商的各种因素之外,还要考虑冲突综合消解以及与应用领域有关的属性等。

3. 协商处理

协商处理是对单个协商方及协商系统、协商行为的描述及分析,包括协商算法和系统分析两部分内容。协商算法用于描述 Agent 在协商过程中的行为,包括通信、决策、规划和知识库操作等。系统分析的任务是分析和评价 Agent 协商的行为和性能,回答协商过程中的问题求解质量、算法效率,以及系统的公平性和死锁等问题。

协商协议主要处理协商过程中 Agent 之间的交互,协商策略主要设计 Agent 内的决策和控制过程,而协商处理则侧重于对单个 Agent 和多个 Agent 协商社会的整体协商行为的描述和分析。前两者描述了多个 Agent 协商的微观方面,而后者则刻画了多个 Agent 协商的宏观层。

11.7 小 结

AI 智能体指由大模型驱动,不需要人为干预,能够自主感知环境、独立进行规划决策、并自动调用工具以完成给定目标的智能程序。

智能体可以看作由大脑、感知、行动三个关键部分组成。

智能体具备特性:自主性、适应性、互动性、学习能力。

Agent 可以看作一个程序或者一个实体,它嵌入在环境中,通过传感器感知环境,通过效应器自治地作用于环境并满足设计要求。

Agent 具备以下特性:自主性、反应性、社会性、进化性。

Agent、体系结构和程序之间的关系:Agent=体系结构+程序。

单个 Agent 的结构可以分为:反应式体系结构、慎思式体系结构和复合式体系结构。

针对不同的应用环境,从不同角度提出了多种类型的 Multi-Agent 模型:BDI 模型、协商模型、协作规划模型和自协调模型等。

多智能体系统的体系结构:网络结构、联盟结构、黑板结构。

智能体通信的类型:使用 Tell 和 Ask 通信、使用形式语言通信。

Agent 通信的方式:黑板系统、消息/对话系统。

智能体通信语言:一是分享一个共同内部表示语言的 Agent,它们无须任何外部语言就能通信;二是无须做出内部语言假设的 Agent,它们共享英语子集作为通信语言。

MAS 中的协调是指多个 Agent 为了一致和谐的方式工作而进行交互的过程。主要有 4 种协调方法:基于集中规划协调、基于协商的协调、基于对策论的协调、基于社会规划的协调。

多智能体的协作是指智能体之间互相配合一起工作。主要有 5 种类型:完全协作型、协作型、自私型、完全自私型、协作与自私共存型。

主要协作方法:合同网协作方法、黑板模型协作方法、市场机制协作方法。

合同网是应用最广泛的一种协作方法,类似于人们在商务过程中用于管理商品和服务的合同机制。合同网协作不需要预先规定智能体的角色,智能体在协作过程中的角色可以变化:任何智能体通过发布任务通知书而成为管理者;相反,任何智能体通过应答任务通知书而成为工作者。

黑板模型由知识源、黑板、监控机制三个基本模块构成。

对于异构环境下的大量智能体之间的协作,采用市场机制协作则是较好的解决方案。市场机制的基本思想是针对分布式资源分配的待定问题,建立相应的计算经济,以使智能体间通过最少的直接通信来协调多个智能体之间的活动。系统中只存在两种类型的智能体:生产者和消费者。生产者能够提供服务,即将某一商品转换为另一商品;消费者能够进行商品交换。智能体以各种价格对商品进行投标,但所有的商品交换都以当前市场价格进行,每一智能体通过投标以便获得最大的利益和效用。

协商是 MAS 实现协调、协作、冲突消解和矛盾处理的关键环节,可以概括为协商协议、协商策略和协商处理三个方面。

协商协议用于处理协商过程中协商方之间的交互和作用,是交易双方交互的规则。

一条协商通信消息:(<协商原语>,<消息内容>)

协商协议的形式化表示方法通常有三种:巴科斯范式表示、有限自动机表示和语义表示。

协商策略是 Agent 选择协商协议和通信消息的策略。可以分为 5 类:单方让步策略、竞争型策略、协作型策略、破坏协商策略和拖延协商策略。竞争型和协作型策略才是有意义的。竞争型策略一般是指协商参与者坚持自己的立场,在协商过程中表现出竞争行为,使协商结果向有利于自身利益方向发展的协商对策。协作型策略则是指协商各方都从整体利益出发,在协商过程中互相合作,他们采取的协商对策有利于寻找互相能接受的协商结果。

协商处理是对单个协商方及协商系统、协商行为的描述及分析,包括协商算法和系统分析两部分内容。

思考题

11.1 Agent 有哪些类型的结构? 各有什么特点?

- 11.2 多 Agent 系统有哪些类型的结构? 各有什么特点?
- 11.3 多 Agent 系统中为什么需要通信? 目前有哪几种通信方式?
- 11.4 什么是多 Agent 系统的协调? 列举几种协调方法。
- 11.5 什么是多 Agent 系统的协作? 列举几种协作方法。
- 11.6 什么是多 Agent 系统的协商? 列举几种协商方法。
- 11.7 选择一个你熟悉的领域, 编写程序描述 Agent 与环境的作用, 并对环境初始状态、Agent 的结构、类型、工作目标加以说明。

读一读

图灵奖得主、深度学习三巨头之一本吉奥

图灵奖获得者、AI 教父、加拿大蒙特利尔大学教授约书亚·本吉奥 (Yoshua Bengio) 是深度学习领域的先驱之一,与杰弗里·辛顿 (Geoffrey Hinton) 和杨立昆 (Yann LeCun) 共同获得了 2018 年的图灵奖,被业界誉为“深度学习三巨头”。

本吉奥发表的包括“Learning Deep Architectures for AI”“A neural probabilistic language model”两篇经典之作在内的 300 多篇论文,对深度学习的发展起到了巨大的推动作用,与 Geoffrey Hinton、Yann LeCun 两位一起造就了 2006 年起的深度学习复兴。他们三人不仅在人工智能的低谷时期坚持研究,持续地带来新的研究成果,更培养了许多人工智能人才。

本吉奥不仅对前沿技术有着深刻的理解,更对 AI 的发展方向、潜在风险以及社会影响有着独到的见解。本吉奥教授说:“要让电脑能像人类那样思考,或者起码能像人类那样理解世界,我们现在离那一步还太远,但是人工智能现在的发展已经足以对经济和人类的福祉产生巨大的影响。”2023 年 8 月,本吉奥被任命为联合国技术进步科学顾问委员会成员并担任加拿大高级研究院 (CIFAR) AI 主席。2024 年,本吉奥入选《时代》杂志全球 100 位最具影响力人物年度榜单。

第 12 章 人工智能在游戏设计中的应用

人工智能游戏的快速发展,为计算机游戏产业提供了新的机遇,目前人工智能技术已经成为优秀计算机游戏开发中不可缺少的部分。

本章首先介绍人工智能游戏的概念以及基本的游戏人工智能技术,然后介绍游戏角色的指导与运动、追逐与躲避、群聚、路径搜索等智能游戏角色设计的基本技术,还介绍智能游戏开发方法与开发工具,最后简单介绍扫雷机智能游戏开发。

12.1 人工智能游戏

继 2013 年“Playing Atari with Deep Reinforcement Learning”发表以后,Google DeepMind 的 alphaGo 在围棋上的 AI 突破又一次震惊了世界!其实,从 1950 年香农教授提出为计算机象棋博弈编写程序开始,人工智能游戏一直就是人工智能技术应用的热点领域,被誉为人工智能界的“果蝇”,推动着人工智能技术的发展。

计算机游戏(computer game)始于 1958 年的《两人网球》(Tennis for Two)游戏。但直到 20 世纪 70 年代,Atari 公司成功开发 Pong 打砖块游戏,才使更多人重视计算机游戏开发,迅速发展成为新兴的游戏产业。人工智能技术的快速发展,为计算机游戏业提供了新机遇。无论玩家是在任天堂的 Wii 游戏机上与马里奥赛车的车手比赛,还是用微软的 Xbox 360 游戏手柄在 Halo 3 上与外来入侵者对抗,人工智能技术是目前优秀计算机游戏开发中不可缺少的部分。角色的智能水平是一款游戏可玩性的决定因素之一,也是游戏开发中需要着重考虑的问题。

应用人工智能技术设计的游戏称为人工智能游戏(AI game),或简称为智能游戏。

人工智能游戏软件给人以某种程度智能的感觉,让玩家感觉更“好玩”,更能令人沉浸其中,成为计算机游戏产品能否畅销的一个决定性因素。

人工智能游戏软件通过分析游戏场景变化、玩家输入获得环境态势的理解,进而控制游戏中各种活动对象的行为逻辑,并做出合理决策,使它们表现得像人类一样智能,旨在提高游戏娱乐性、挑战智能极限。人工智能游戏是结果导向的,最关注决策环节,可以看作“状态(输入)”到“行为(输出)”的映射,只要游戏能够根据输入给出一个看似智能的输出,那么就认为此游戏是智能的,而不在乎其智能是怎么实现的(whatever works)。

游戏开发者利用人工智能让无数的角色看起来好像是有智慧的生命一样,表现出不同的人格特质,或者呈现出人类特有的情绪或脾气,从而吸引玩家。

对游戏的智能性,具有代表性的几种观点是:

① 如经典游戏《小精灵》(Pac-Man)里的魔鬼、Unreal 第一人称射击游戏(FPS)里的虚拟战友以及其他游戏角色,它们看起来都是具有智能的生命。这种游戏可以认为是有智能的。

② 有人将游戏中的路径搜索、碰撞检测等,也列入人工智能游戏的范畴。

③ 游戏角色从简单的追逐、闪躲、移动,到复杂的神经网络和遗传算法应用等,能够体现角色行动的“自主性”,则说明游戏具有智能性。

人工智能游戏并不是特别关心智能体是否表现得像人类一样,而是更加关心人工智能游戏的智能极限——能否战胜人类的领域专家,例如,Waston 在智能问答方面战胜了“Jeopardy!”节目的超级明星 Ken Jennings 和 Brad Rutter; AlphaGo 在围棋上战胜了多位顶尖棋手。

目前最有影响的人工智能游戏比赛是国际机器人图灵测试比赛。比赛内容是让程序员开发出一个软件“机器人”来控制游戏人物,使其被一群专业玩家误认为是真人。机器人图灵测试是由艾伦·图灵(Alan Turing)设计的图灵测试的一个变种,使机器仅仅在文本谈话下,令评委感觉它是真人。

澳大利亚佩斯的埃迪斯·科文大学计算机与信息学院菲利普·亨斯顿(Philip Hingston)说:“机器人图灵测试对于游戏当中的人工智能非常重要,通过向游戏玩家提供更有对手,人工智能可以使游戏变得更加好玩。这也对广义的人工智能意义重大,因为它强调了人工智能的核心问题,即人类的智能如何与计算机的智能联系起来?”

史诗游戏公司首席程序员史蒂夫·波巨(Steve Polge)认为创建一个好的机器人是极具挑战性的。该公司开发了虚幻竞技场游戏。他说:“你不能总要求人工智能可以像人一样,因为人可以成为非常惹人厌的、可憎的对手。”相反,波巨认为,开发人员是在开发“能够制订出人意料的计划,并给玩家突然袭击或者设定意外挑战的人工智能,这无疑会使游戏变得更好玩。”

得克萨斯大学的计算机科学与神经科学教授里斯托·米库莱宁(Risto Miikkulainen)是机器人图灵测试参赛者之一,他试图探索合理的人机融合,设计机器人可以快速学习。他说:“当人们玩游戏时,他们适应得很快,所以在制造机器人时,你不能让他们 100%精确,因为适应性是不精确的。”

包括《模拟人生》(The Sims)和《孢子》(Spore)在内的畅销仿真游戏开发者威尔·怀特(Will Wright)希望,机器人图灵测试能够鼓励人工智能研究人员实现人最难以捉摸的特性——情感。“机器响应成为我们生活中无处不在的部分,但它们总不那么令人满意。”怀特说,“所以,在人工智能中,认知我们感情的维度是一项有趣的任务。”

从商业角度,游戏产业主要关心的是采用新技术所带来的效益。现代游戏的制作都是十分庞大的工程,目前还不能利用人工智能来创作整个游戏。人工智能程序还不是游戏开发的重中之重。游戏开发人员还没有足够地重视人工智能技术在游戏开发中的应用,他们主要的注意力仍然放在图画的质量上,包括三维、纹理映射和实时程序渲染语言等计算机图形学技术。实际

上,用花来提高图画质量上的人力和财力,可以开发出更好的智能游戏,特别是计算机图形学总有一天将不再成为游戏技术发展的驱动力。因为效用递减规律将使普通玩家无法区分当前的图形技术与以前的图形技术的差别。而人工智能技术将成为游戏革新新驱动力,带来新风格的游戏,为主流玩家提供令人兴奋的效果,提供有助于市场竞争的新的计算机游戏产品。

自香农 1950 年发表计算机象棋博弈编写程序的方案以来,人工智能游戏已经走过了半个多世纪,取得的成果数不胜数,下面罗列了人工智能游戏的重大里程碑,使读者能够把握人工智能游戏的研究现状,为今后的研究方向给予启示,具体如表 12.1 所示。

表 12.1 人工智能游戏里程碑表

年份	名称	作者	描述及意义
1950	计算机象棋博弈编写程序的方案	克劳德·艾尔伍德·香农	机器博弈的创始;奠定了计算机博弈的基础
1951	Turochamp	艾伦·麦席森·图灵	第一个博弈程序;使用了博弈树
1956	西洋跳棋	阿瑟·塞缪尔	第一个具有自学习能力的游戏;使用了强化学习算法,战胜康涅狄格州的西洋跳棋冠军
20 世纪 60 年代	国际象棋弈棋程序	约翰·麦卡锡	第一次使用 Alpha-beta 剪枝的博弈树算法,使得博弈树的搜索深度增加一倍
1980	Pac-man	Namco 公司	第一次使用了规则与动作序列及随机决策等技术
1989	Simcity	Maxis 公司	第一次使用了人工生命、有限状态机技术
1990	围棋弈棋程序	艾布拉姆森	第一次将蒙特卡洛方法应用于计算机国际象棋博弈的盘面评估
1996	Battlecruiser 3000AD	3000AD 公司	第一次在商业游戏中使用神经网络
1996	Creature	NTFusion 公司	第一次引入 DNA 概念、遗传算法
1997	Deep Blue	IBM 公司	第一个达到人类国际大师水平的计算机国际象棋程序;使用了剪枝的博弈树技术,以 3.5 : 2.5 战胜世界棋王卡斯帕罗夫
2011	Watson	IBM 公司	第一次在智能问答方面战胜人类;使用了 Deep QA 技术,战胜了 Jeopardy! 节目的超级明星 Ken Jennings 和 Brad Rutter
2013	玩“ Atari 2600 ”的程序	DeepMind 团队	第一个“无须任何先验知识”“自学习”“可迁移”的游戏引擎;使用了 CNN+Q-learning 的 DRN 技术,输入为“原始视频”和“游戏反馈”,无须任何人为设置,可从“零基础”成为 7 款“ Atari 2600 ”游戏的专家

续表

年份	名称	作者	描述及意义
2015	AlphaGo	Google DeepMind 团队	第一个在全棋盘(19×19)无让子的情况下战胜人类专业棋手的“围棋程序”;使用了 SL(有监督学习)+RL(强化学习)+MCTS(蒙特卡洛树搜索)方法,5:0 战胜了欧洲围棋冠军樊麾
2016	AlphaGo	Google DeepMind 团队	在韩国举行的比赛中,AlphaGo 以 4:1 战胜世界围棋冠军李世石
2017	AlphaGo Master	Google DeepMind 团队	运行于谷歌云端的单个 TPU 围棋软件,在乌镇举行的比赛中以 3:0 完胜世界排名第一的柯洁

12.2 游戏人工智能

12.2.1 游戏人工智能的概念与分类

适合于游戏开发的人工智能技术称为游戏人工智能(game AI)。

与一般的人工智能技术不同,游戏人工智能算法不一定要满足通用性,只要能够使角色的行为在某些场合内合理就行了。实际上,即使出现了一些人工智能错误,只要不是很明显,而且不降低游戏的娱乐性,玩家不会太在乎,甚至给玩家带来意想不到的兴奋。而对于医疗诊断等专家系统,这些错误就不能存在了。

游戏人工智能分为定性(deterministic)和非定性(nondeterministic)两类。

定性技术是游戏人工智能的基础。用定性技术设计的角色行为是特定的、可预测的。游戏中的角色分为玩家和非玩家两种角色。例如,可以用定性技术设计一个非玩家角色(NPC),沿着坐标轴前进,向某一个目标点移动,直到该角色坐标值与目标点的坐标相同,就停止前进。定性技术容易实现,理解方便,也便于游戏软件的调试和测试。在用定性技术设计游戏时,游戏开发者必须考虑所有的可预测行为,而且无法帮助非玩家角色的学习或演化。玩家只要重复玩几次用定性技术设计的游戏,就可知道 NPC 的行为,这样,玩家就会失去玩这些游戏的兴趣,从而影响游戏软件的“生命”。

非定性技术是定性技术的一种提高。用非定性技术设计的角色行为具有某种程度的不确定性和不可预测性。在用非定性技术设计的游戏中,NPC 能够学习到玩家的作战行为,并推出新行为,甚至引发突现行为。例如,用非定性技术设计海里的一群鱼,鱼群总体上都会跟着最前面带路的那条鱼游动,而且不会出现鱼和鱼之间的穿透现象,这是属于 NPC 群聚行为,使鱼群表现得更加逼真。由于非定性的不可预测性,会给游戏软件的调试和测试带来一定的难度,但也增加玩家对游戏的兴趣,延长游戏软件的“生命”。

未来的游戏人工智能越来越注重非定性技术的研究与应用。如 Creatures, Black&White, Battlecruiser3000AD, Dirt Track Racing, Fields of Battle, Heavy Gear 等一些主流游戏,都采用了非定性技术,并具有一定的学习功能。所有非玩家角色的行为不再是事先安排好的,而是随着游戏的进展,NPC 从玩家那里不断学习,使玩家难以预测 NPC 的行为,从而使游戏更具挑战性,也扩展了游戏的生命周期。

成功的游戏软件应该采用定性技术和非定性技术相结合的方法。用定性技术解决软件的一部分调试和测试问题,用非定性技术增强软件的智能性,赋予软件更强的生命力和挑战性。任何一种非定性技术,即使功能很强,也不可能解决游戏中的所有问题,体现整个软件的智能性。所以,在使用非定性技术时,恰当地运用定性技术,对软件开发的周期、软件质量等环节都是非常有益的。

目前,在人工智能技术中,采用神经网络、遗传算法、贝叶斯技术、有限状态机等用来实现游戏中的非定性行为。

12.2.2 基本的游戏人工智能技术

目前,基本的游戏人工智能技术有以下几种。

1. 搜索技术

搜索技术在现代游戏中应用非常广泛,特别是用于路径规划。当前棋类游戏几乎都使用了搜索的方式来完成决策的,其中最优秀的是搜索树、蒙特卡洛搜索树。搜索的游戏人工智能更关注的是预测和评估,决策的制定只是挑选出预测的最好方式即可。在追捕游戏中,只要追逐角色与任一非追逐角色相邻,所对应的状态就是目标状态。对于简单的情况,广泛采用宽度优先和深度优先等非启发式搜索算法。但这些算法需要指数级内存来存储游戏状态的实例。在游戏设计中广泛应用 A* 算法,例如,在追捕游戏中以两点间欧氏距离为启发函数。A* 算法能够保证以最少的搜索找到最优的路径。现代游戏设计中特别需要研究对抗搜索等方法。

2. 遗传算法

遗传算法是一种随机优化搜索技术,它模拟达尔文进化论的物竞天择原理,不断从种群中筛选最优个体,淘汰不良个体,使得系统不断自我改进。遗传算法可以使得游戏算法通过不断的迭代进化,可以从一个游戏白痴进化成为游戏高手,是一个自学习的算法,无须任何人类知识的参与。遗传算法已经广泛地应用于智能游戏设计。例如,游戏设计中经常需要为某个角色寻找最优路径,往往只考虑距离是远远不够的。游戏设计中利用了一个 3D 地形引擎,需要考虑路径上的地形坡度。当角色走上坡路时应该慢些,而且更费油料。当在泥泞里跋涉时应该比行驶在公路上慢。采用遗传算法进行游戏设计时,可以定义一个考虑所有这些要素的适应度函数,从而在移动距离、地形坡度、地表属性之间达到较好的平衡。可以为游戏中不同的地表面创建不同的障碍值或者惩罚值并加入适应度函数。如果道路泥泞则惩罚值大,该道路总的适应度就小,选择这条路径的可能性就小。当然,如果这条路径比较短,使得适应度增加,选择这条路径的可能性变大。对地形坡度的处理也是类似的。最终路径的选择是所有因素的

折中考虑。NPC 要平衡好这些因素是比较困难的,但最后一般都会像真人选择路径那样能够为各种地形找出最优路径,而不是仅仅找到距离最短的路径。

3. 模糊逻辑

在游戏设计中广泛应用模糊逻辑方法。例如,用模糊逻辑控制队友,或者其他非玩家角色,能够实现平滑运动,看上去更自然。在战争游戏中,计算机军队经常得配置防卫兵力,以抵抗潜在威胁的敌军。计算机军队可以根据敌军的距离以及规模等用模糊逻辑评估玩家对计算机军队即非玩家的威胁。其中,距离可以用“很近”“较近”“较远”“很远”等表示;规模可以用“零星”“少量”“中等”“大量”等表示;而威胁程度可以用“无”“小”“中”“大”等表示。可以根据玩家或者非玩家角色的体力、武器熟练度、被击中的次数、盔甲等级等因素,将玩家或者非玩家的战斗能力分为“弱”“较弱”“一般”“较强”“很强”五个等级。

4. 神经网络

神经网络是基于生物大脑和神经系统中的神经连接结构的一系列机器学习算法的总和。在具体使用中,通过反复调节神经网络中互连结点之间的参数值(权重)来获得针对不同学习任务的最优和近似最优反馈值。本章最后将介绍一个用神经网络控制扫雷机的实例。

DRN 是 Google DeepMind 在“Playing Atari with Deep Reinforcement Learning”文章中结合深度学习和强化学习形成的神经网络算法,旨在无须任何人类知识,采用同一算法就可以在多款游戏上从游戏白痴变成游戏高手。首先使用数据预处理方法,把 128 色的 210×160 的图像处理成灰度的 110×84 的图像,然后从中选出游戏画面重要的 84×84 的图像作为神经网络的输入;接着使用 CNN 自动进行特征提取和体征表示,作为 BP 神经网络的输入进行有监督学习,自动进行游戏策略的学习。

Google DeepMind 的 AlphaGo 使用价值网络来评价棋局形式,使用策略网络来选择棋盘着法,使用蒙特卡洛树来预测和前瞻棋局,试图冲破人类智能最后的堡垒。策略网络使用有监督学习+强化学习共同训练获得,而价值网络仅使用强化学习训练获得,通过这两个网络结构总结人类经验,自主学习获得对棋局的认知;然后使用蒙特卡洛树将已经学得的知识应用于棋局进行预测和评估,从而制定着法的选择。

5. 一阶谓词逻辑

在游戏设计中,用一阶谓词逻辑描述变化世界的方法,称为情景演算(situation calculus)。用一阶谓词逻辑来计算给定情况下智能主体的应有动作,用自动推理来决定达到最优状态所需采取的动作序列。正是由于需要智能角色去思考变化的世界,而情景演算正好适合描述变化的事物,因此得到较为广泛的应用。但它不适用于对性能要求很高的实时游戏中,而且用逻辑语言来描述复杂游戏是比较困难的。

6. 专家系统

专家系统用于模拟专业玩家的行为,游戏开发人员编写知识库控制角色的行为。尽管智能游戏的知识库在表达上不需要像其他专家系统的知识库那样复杂,但随着游戏的日益复杂化,专家系统越来越难以建立。现在少数游戏专家系统已引入机器学习。机器学习将在未来

智能游戏开发中得到越来越广泛的应用。

7. 机器学习

1956 年阿瑟·塞缪尔的西洋跳棋程序仅通过强化学习算法让自己和自己对战就可以战胜康涅狄格州的西洋跳棋冠军。强化学习来源于行为主义学派,它通过 Agent 与环境的交互获得“奖惩”,然后趋利避害,进而做出最优决策。强化学习有着坚实的数学基础,也有着成熟的算法,在机器人寻址、游戏智能、分析预测等领域有着很多应用。成熟算法有值迭代、策略迭代、Q-learning、动态规划、时间差分等学习方法。

分类是有监督学习方法,旨在具有标签的数据中挖掘出类别的分类特性,也是游戏人工智能的重要方法。有些玩家就是通过学习前人经验成为游戏高手的,棋类游戏更是如此。经过几千年的探索,人类积累了包括开局库、残局库、战术等不同的战法来指导玩家游戏。而数据挖掘就是在数据中挖掘潜在信息,并总结成知识然后指导决策的过程。Google DeepMind 的围棋程序 AlphaGo 就使用了有监督学习训练策略网络,用以指导游戏决策;它的 SL 策略网络是从 KGS GO 服务器上的 30 000 000 万棋局记录中使用随机梯度上升法训练的一个 13 层的神经网络,在测试集上达到 57.0% 的准确率,为围棋的下子策略提供了帮助。

8. 多智能体

通过多个彼此竞争和协作的智能主体描述角色之间的交互。游戏中存在的众多角色彼此既有竞争也有合作,因此可以用多智能体来自然地产生智能行为。

9. 人工生命

由简单的个体行为组合成复杂模式的研究,称为人工生命 (artificial life)。这是多智能体系统中的一种,着重研究如何为虚拟环境中的智能主体赋予某些生物体的共性。游戏 The Sims 和 SimCity 的成功证明了人工生命技术的有效性和娱乐价值。例如,用人工生命设计群聚,控制对象的智能化运动,协调多个智能主体的动作,使它们在整体上看起来像逼真的动物群。

10. 基于范例的推理

这种技术分析数据库中存放的(历史)输入数据和相应的最优输出结果,然后通过对比现有输入数据和历史数据来推知输出结果。这种技术模拟了人们在处理新的情况时参考以前相似经历的做法。

11. 有限状态机 (finite state machine, FSM)

有限状态机是表示有限个状态以及在这些状态之间进行转移和动作等行为的特殊有向图,可以通俗地解释为“一个有限状态机是一个设备,或是一个设备模型,具有有限数量的状态”。它可以在任何给定的时间根据输入进行操作,使得从一个状态变换到另一个状态,或者是促使一个输出或者一种行为的发生。这是一种简单的基于规则的系统,它包含了有限个“状态”和状态之间的“转移”,彼此连成一个有向图。有限状态机在每一时刻都只能处于某一状态。在智能游戏设计中,通过相应不同游戏状态,并完成状态之间的相互转换可以实现一个看似智能的游戏智能引擎,增加游戏的娱乐性和挑战性。有限状态机把游戏对象复杂的行为分

解成“块”或状态进行处理。首先接收游戏环境的态势和玩家输入;然后提取低阶语义信息,根据每个状态的先决条件映射到响应的状态;接着根据响应状态的产生式规则生成动作方案;最后执行响应动作序列并输入游戏,进入下一步循环。有限状态机方法是计算机游戏机理的一种简单实现:根据规则人为将原始数据映射到“状态”完成“特征工程和识别”,根据产生式系统将“状态”映射到响应的动作完成“决策制定”。有限状态机在智能游戏设计中应用最为广泛,其主要原因是游戏中的非玩家角色常可以设计为含有单个状态变量,而变量值可以表达为有限状态机当前结点的形式。结点的输入和输出则驱动角色在那个状态下的行为。有限状态机的缺点是当状态数目和状态转移数目增加时复杂度大大增加。

12. 决策树(decision tree)

决策树表达一系列产生式规则 if-then 形式的条件判断,容易转化为一组产生式规则。在决策过程中,从决策树的根结点输入一组数据,在每个分叉处根据某个输入值选择其中一个子结点,以此类推。国际象棋和西洋双陆棋等许多棋类游戏中成功采用了游戏树及其搜索方法。决策树在计算机游戏中经常被用来表达控制器。决策树的编程容易实现。非专业程序设计人员可以通过图像用户界面方便地建立与维护决策树。如果每个叶结点所对应的不是单一的行动选择,而是一个所有可能行动的概率分布函数,那么,就成为随机决策树,可用于表达随机化控制器。

13. 置信网络

置信网络采用了概率理论来解决现实世界中的不确定性和不完全知识等问题,是描述不同现象之间内在因果关系的工具。还可以用于推导现实世界的状态和预测各种动作的可能结果。这种技术特别适用于解决智能游戏中涉及的许多子问题。

人工智能游戏设计中或多或少地采用了上述技术,所获得的效果也不尽相同。在开发期限紧张和资源有限的情况下,很多游戏开发人员都倾向于采用基于规则的系统,因为编写、理解和调试基于规则的智能游戏程序比较容易。

12.3 游戏中的角色与分类

游戏中的活动对象分为两类。一类是背景中的活动对象,如天上飘着的云、飞过的鸟等。这类对象的造型和行为要显得逼真也不容易,需要掌握 2D 或者 3D 图形和动画技术,还需要有艺术修养。但它们在游戏中无须人工干预,变化也不多,控制的逻辑比较简单。

另一类活动对象是游戏中的各种角色,或者称为游戏代理,如虚拟的人、兽、怪物、机器人等。角色的活动方式必须变化多端才行,否则游戏就不好玩,所以需要设计比较复杂的角色控制逻辑。尤其是玩家对手代理的控制逻辑最复杂。例如,要开发一个猫捉老鼠的游戏。假设你是玩家,你的代理是猫,即猫的行为由你操纵,而老鼠的行为则完全需要由计算机程序来控制。当猫不出现时,老鼠必须到处觅食或者打洞,以解决生存必需的吃住问题。一旦发现有猫,则立即躲进洞里。如果附近没有洞,则要立即逃窜。逃窜的方向应该和猫是反方向的。如

果途中遇到障碍物挡住了去路,则要改变方向,可以向左或者向右,但不应该回头跑,除了前面是死胡同。老鼠能遵循这样的逻辑来行动,就是游戏编程中为老鼠设计的智能。虽然游戏里的猫也需要人工智能,但很简单,那就是听话,能够听从你用键盘或者鼠标进行的指挥。

所有角色扮演类游戏都需要智能。越是好玩的游戏越是需要复杂的智能。但并不是所有的游戏都需要人工智能。例如,Windows 提供的接龙和挖地雷等游戏就没有人工智能问题。网上提供的两人对弈的象棋、围棋、军棋类游戏也不需要人工智能技术。但一旦要求机器与人对弈,那就需要很高的智能了。

游戏角色的智能水平是一款游戏可玩性的决定性因素之一,因而也是游戏开发中需要考虑的重要问题。人工智能技术能够实现智能角色,增强游戏的体验并改善游戏的可玩性。

游戏中的角色除了根据角色的属性分为玩家和非玩家两种类型,还可以有下面几种分类方法。

① 根据运动方向与朝向的相关性,游戏角色分为靠转向力改变的角色和自由运动的角色两种类型。靠转向力改变运动方向的角色,如汽车、飞机之类的物体,它们速度向量的方向必须与其朝向相同或相反,速度方向的改变只能靠施加在头部或尾部的转向力(steering force)实现。特别是有的角色要求当速度大小不为0时,施加转向力才有效。例如,静止的汽车,对其方向盘的操作是毫无效果的。

② 根据角色的智能性,游戏角色分为智能性角色和无智能性角色两种。角色运动的结果主要是位置和朝向的变化,而运动的方式可有多种。对玩家来说,通过操控键盘、鼠标或游标等来实现玩家角色的运动,如行走、跑步、转向、坐蹲等。对智能非玩家角色来说,通过预置各种运动算法,当运动条件和环境符合时,自动产生运动方式。对无智能性非玩家角色,则不产生任何运动,只能靠场景切换或摄影机移动来实现隐藏或现身。

12.4 智能游戏角色设计基本技术

角色的智能性具体表现在指导与运动、追逐与躲避、群聚、路径搜索、智能搜索引擎等方面,下面做个简单介绍。

12.4.1 游戏角色的指导与运动

有两种方法指导角色行为:预定义行为(predefined behavior)和目标导向行为(goal-directed behavior)。区分预定义行为和目标导向行为的依据是:角色是否唯一地选择行动。

预定义行为指导角色行为方法中,角色的所有行为都是动画师和程序设计人员预先设计的。这里讨论的主要是如何处理预先并不完全确定的角色行为的技术。两种比较流行的方法是反应行为规则(reactive behavior rules)和分层有限状态机(hierarchical finite state machine, HFSM)。下面简单介绍反应行为规则。

当角色的行为仅仅取决于当前外部环境的感知时,称为反应行为。在反应行为中,角色不

记忆以前经历的情景,没有自己内部情景的表达,因此,无论角色收到的信号顺序如何,只要是同样的刺激信号,其反应行为就是一样的。

反应行为规则是目前最普及的生成角色行为的方法,用一些简单的产生式规则就能够产生复杂的行为。例如,下面简单的左转规则:

若 有障碍在前面 则 向左转 否则 直走
IF blocked ahead THEN turn-left ELSE go-straight

角色没有必要知道关于迷宫的知识,只要依据上面的规则执行,就能够找到走出迷宫的道路。

由于行为规则是预先确定的,所以一旦选定了行动,就不能再改变。许多情况下,角色依据领域知识,很快可以预测所选的行动是不合适的。但是,不管是否有其他更合适的行动方案,自主角色都必须按照当前情景预定义行为规则行动。因此,像分层有限状态机为角色保留了某些简单的内部状态信息,设计控制器就比较容易。

在游戏设计中,许多角色选择行动是非确定性的,有多个方案可以选择。当角色选择了某个行动方案后,如果后来看出这个方案不能达到期望的目标,可以返回选择其他方案,直到选出合适的方案。非确定性选择行动的角色容易接受指导,通过给角色指定目标,使角色能够利用自己的领域知识,了解它选择的行动方案是否能够达到目标。但非确定性选择行动的响应速度比较慢。游戏设计时应该折中考虑预定义行为和目标导向行为指导角色的方法。因此,指导角色的第三种方案是将预定义行为和目标导向行为结合起来。

12.4.2 游戏角色的追逐与躲避

在游戏中,追逐与躲避是角色最普遍的行为。它与随机移动一起构成了早期游戏中主要使用的运动模式。

1. 随机移动

随机移动能够产生角色漫无目的徘徊的感觉。对于自由运动的角色,可以不断地产生随机的速度向量;对于靠转向力来改变速度方向的角色,可以不断产生随机的转向力。若使用完全的随机数来控制角色的移动,行动看起来不稳定,所以应该保持随机过程的一致性,可采用噪声函数法、投射目标法,也可以记录从前走过的路,避免走回头路等来体现随机性。

2. 追逐与躲避

追逐与躲避是功能相反的两种运动模式,在游戏中到处可见。例如,在太空战机射击游戏或角色扮演游戏,游戏中的非玩家角色都有机会试着追逐或躲避玩家角色;在角色扮演冒险游戏中,也想要个巨人或某种可爱的生物,追着你的玩家角色;在第一人称射击游戏和飞行模拟游戏中,需要算出导弹的轨迹,攻击玩家或玩家的飞行器等。对于任何一种情况,都需要一种逻辑手段,使角色进行追逐或躲避。

3. 躲避障碍物

玩家与非玩家角色在游戏中都应该符合现实场景的客观规律,不能出现互相穿透现象。

如玩家穿过非玩家角色、玩家走出虚拟场景、玩家穿越地面等。对于这些现象,在设计过程中,可以理解为是追逐与躲避的一种特例。躲避障碍物的方法往往采用传感器、球包围盒等方法,自动检测玩家与物体的距离,当达到测定距离时,可以自动实现玩家不能靠近障碍物。

使用传感器来判断障碍物的基本方法是利用传感器从当前位置沿当前速度向前方一段距离作投射。传感器的末端表示按照当前的速度将要到达的位置。若在这个投射上发生了碰撞,则表示按当前速度行进将会发生碰撞,以此要求角色改变速度。传感器的长度可以与角色的速率成正比。转向力的方向为障碍物表面的碰撞点的法向量的方向,转向力的大小可以与传感器的末端穿进障碍物的深浅成正比。

球包围盒或长方体包围盒是将物体抽象地定义为一个位置坐标和一个碰撞半径来检测角色是否将与物体相碰撞。

12.4.3 游戏角色的群聚

角色的群聚(flocking)行为是基于 Craig Reynolds 提出的 Boids 算法,该算法由三个简单的规则组成。

- ① 聚合(cohesion):每一个成员移向它的邻居成员的平均位置。
- ② 对齐(alignment):每一个成员朝向它的邻居成员的平均朝向。
- ③ 分离(separation):每一个成员与邻居保持一定的距离,以免相撞。

这些规则都可以利用追逐或躲避算法来实现,可以产生非常逼真的群聚效果。

团队中的各个成员所考虑的邻居成员的数量可以通过该成员的视角和视野来控制,只有在视角和视野范围内的成员才予以计算。设置大的视角会使队形向横向排开,而小的视角会使队形纵向排开,形成细长的队列。可以为不同的规则制定不同的视角和视野,这些需要在游戏的实现过程中进行调整。

一个成员的行为结果是上述三个规则计算出的转向力的组合。实际上这三个规则计算出的转向力还可以和由其他行为,如追逐、躲避、跟随首领等计算出的转向力组合。有如下几种组合方法。

① 为每一个转向力设置一个权值,将所有的转向力加权,再截取不超过转向力的最大值。这种方法的缺点是权值难以设定和调整,因为有些转向力的施加是必需的,例如,障碍物躲避是必须的,即使它与群聚的转向力发生冲突,也应该保证角色不与障碍物相撞。若过多增加该转向力的权值,会产生不理想的行为,在两个成员接近时行为不稳定,且容易使团队分离。

② 在①的基础上为每个转向力增加一个优先级。除了按照权值组合外,每一次更新按照转向力的优先级的高低顺序组合,若组合的结果大于转向力的最大值,则进行截取并忽略低优先级的转向力。

③ 不设定权值,而是为每个转向力增加一个概率和一个优先级。每一次更新只有一个转向力起作用。首先考虑优先级高的转向力,只有在概率命中,并且结果不为0才进行组合;否

则,考虑下一个优先级的转向力。在这种方法中,由于每一次更新只计算一个转向力,减少了计算量,但却是以降低精度为代价的。

12.4.4 游戏角色的路径搜索

路径搜索是智能游戏软件中最基本的问题之一。有效的路径搜索方法可以让角色看起来很真实,使游戏变得更有趣味性。

在简单的情况下,有以下几种路径搜索算法。

① 在不考虑躲避障碍物的情况下,路径搜索算法即为追逐算法。

② 若考虑躲避障碍物,则随机移动;如果直线上没有障碍物,则沿直线向目标移动。这种方法在开阔的场景,且障碍物数量较少、体积较小的情况下(如只有少量的树木)是可行的。

③ 当场景中有体积较大的障碍物,则从追逐者位置向目标作投射得到一条直线,然后沿直线向目标移动。当碰到障碍物时,则沿着障碍物的边缘移动,直到又回到直线上,继续沿着直线移动。但当碰到障碍物的内转角时,不太适合采用这种方法。

④ 面包屑寻路法是记录目标走过的路径,让追逐者沿着这条路径移动。记录目标移动路径的方法是在目标走过的地方留下玩家看不见的记号(面包屑),追逐者可以按照不同的面包屑顺序移动。采用这种方法可能会出现追逐者走回头路的现象。

⑤ A* 搜索算法是一种启发式搜索策略,能保证在任何起点与终点之间找到最佳路径。由于 A* 搜索算法相对比较复杂,要求 CPU 做大量的计算,因此,现在还不是游戏软件开发中最常用的路径搜索算法。特别是当 CPU 功能不太强,尤其是解决多角色游戏的路径选择问题, A* 算法不是最佳选择,否则会影响游戏效果。由于路径的类型很多,寻求路径的方法应与路径的类型、寻径的需求有关, A* 算法不一定适合所有场合。例如,如果起点和终点之间没有障碍物,有明确的视线,就没有必要使用 A* 算法。

12.4.5 智能搜索引擎

在游戏开发中,核心技术是构架游戏引擎,它是决定游戏质量的关键。游戏引擎就像赛车的引擎。引擎是赛车的核心,决定着赛车的性能和稳定性。赛车的速度、操作感这些直接与车手相关的指标都是建立在引擎的基础上的。游戏也是如此,玩家所体验到的剧情、关卡、美工、音乐、操作等内容都是由游戏引擎直接控制的,它扮演着中场发动机的角色,将游戏中所有元素捆绑在一起,在后台指挥它们同时、有序地工作。简单地说,引擎就是用于控制所有游戏功能的主程序,从计算碰撞、物理系统和物体的相对位置,到接受玩家的输入,以及按照正确的音量输出声音等。

从狭义上讲,游戏引擎就是指在对部分通用技术细节进行整理和封装的基础上,形成一个面向游戏应用的应用程序接口(API 函数),使得游戏开发人员不必再关心底层技术的实现细节,只需要调用游戏引擎中相关的 API 函数。同样在游戏里,剧情的进行、形形色色的角色衬托、各种场景的交换,也都是游戏引擎事先就约定的一种模式或大体的框架,然后依靠一些复

杂的数据库来组织完成。

游戏引擎从 20 世纪 90 年代初开始,经历了诞生、发展、成熟三个阶段。在每个阶段,都有优秀的游戏引擎产生,它们组成了游戏引擎的演化史。从 2000 年开始,游戏引擎朝着两个不同的方向分化。一是通过增加更多的叙事成分和角色扮演成分以及加强游戏的人工智能来提高游戏的可玩性,如《半条命》《神偷》《杀出重围》等;二是向着纯粹的网络模式方向发展,如 Id Software 公司在 Quake 2 出色的图像引擎的基础上加入更多的网络成分,推出了一款完全没有单人过关模式的纯粹的网络游戏——《雷神之锤 3 竞技场》,它与 Epic 公司稍后推出的《虚幻竞技场》一同成为引擎发展史上的一个转折点。

在人工智能方面真正取得突破的游戏是 Looking Glass 工作室的《神偷:暗黑计划》。这个游戏的故事发生在中古年代。玩家扮演一名盗贼,任务是进入不同的场所,在尽量不引起别人注意的情况下窃取物品。《神偷》采用的是 Looking Glass 工作室自行开发的 Dark 引擎。Dark 引擎在图像方面比不上《雷神之锤 2》或《虚幻》,但在人工智能方面,它的水准却远远高于《雷神之锤 2》和《虚幻》。这个游戏中的角色懂得根据声音辨认敌人的方位,能够分辨出不同地面上的脚步声,在不同的光照环境下有不同的视力,发现同伴的尸体后会进入警戒状态,还会针对玩家的动作做出合理的反应,玩家必须躲在暗处不被敌人发现才有可能完成任务。这些在以前那些纯粹的杀戮游戏中是根本看不到的。此后的绝大部分第一人称射击游戏都或多或少地采用了这种隐秘的风格,包括《荣誉勋章:盟军进攻》(Medal of Honor: Allied Assault)。

12.5 智能游戏开发方法与开发工具

12.5.1 智能游戏开发方法

实现游戏智能有两种不同的方式。一种是采用传统的编程技术,使系统呈现智能的效果,而不考虑所用方法是否与人或者动物所用的方法相同。这种方法称为工程学方法(engineering approach),它已经在文字识别、电脑下棋等许多领域应用。另一种是模拟法(modeling approach),它不仅要看效果,还要求实现方法也和人类或动物机体所用的方法相同或相类似,如遗传算法、神经网络等。

为了得到相同的智能效果,两种方法通常都可以使用。采用前一种方法,需要人工详细规定程序逻辑。因此,如果游戏简单,这种方法还是很方便的。但如果游戏复杂,角色数量和活动空间增加,相应的逻辑复杂度就会以指数增长,人工编程就会非常复杂,容易出错。而采用后一种方法,设计者不需要详细规定程序逻辑,而是给每个角色设置一个智能模块来进行控制。这个智能模块开始什么也不懂,就像初生婴儿那样,但他能够学习,能够渐渐地适应环境。利用这种方法来实现游戏智能,无须对角色的活动规律作详细规定,应用于复杂问题时,通常会比前一种方法更省力。

12.5.2 智能游戏开发工具

选择合适的游戏开发工具,对于游戏功能的实现、游戏的复杂度、游戏的智能性、游戏的开发效率、游戏功能的可扩充性和移植性,以及开发者使用工具的能力、开发付出的工作量、游戏软件实现的难易程度等,都起着至关重要的作用。

游戏开发工具分为创作工具类和编程语言类两大类。

创作工具类是由软件开发商开发的集各种功能在内的专业软件,主要采用“拖、拉、放”的形式进行开发。如 Virtools、PRG Maker XP、FPS Creator、Vega、Game Jackal Pro、ZD Soft Game Recorder、Game Editor 等开发工具。

编程语言类主要利用各种高级语言,通过编程实现游戏的开发,如 VC++、C、Java、J2ME、VC. NET、Delphi、VB 等。

选择游戏开发工具应综合考虑各种因素,可以选择创作工具类,也可以选择编程语言类,或者是两者的结合。

12.6 扫雷机智能游戏开发

扫雷机工作在一个很简单的环境中,只需要若干扫雷机和随机散布的许多地雷。游戏设计目标是设计一个 BP 神经网络,能够自己演化去寻找地雷。

基本方法是用一个 BP 神经网络控制一个扫雷机,它的权值用遗传算法进行演化,使扫雷机更聪明。

可以把扫雷机看成与坦克一样,通过左右两个能够转动的履带式轮轨来行动。通过改变两个轮轨的相对速度来控制扫雷机前进的速度以及向左或者向右转弯的角度。因此,选择左右两个履带轮的速度作为 BP 神经网络的两个输出。

这里为每个扫雷机装上触觉器,从而能够避开障碍物。这里的触觉器是从扫雷机向外辐射出来的几根线段,如图 12.1 所示。线段的长度和数目都是可以调整的。

在每一帧中都要调用一个函数来检测每个触觉器是否与周围世界的障碍物的边界线相交。这些触觉器检测到的数据输入扫雷机的神经网络。因此,选择控制扫雷机的输入信息是扫雷机的视线向量和扫雷机到最近地雷的向量。

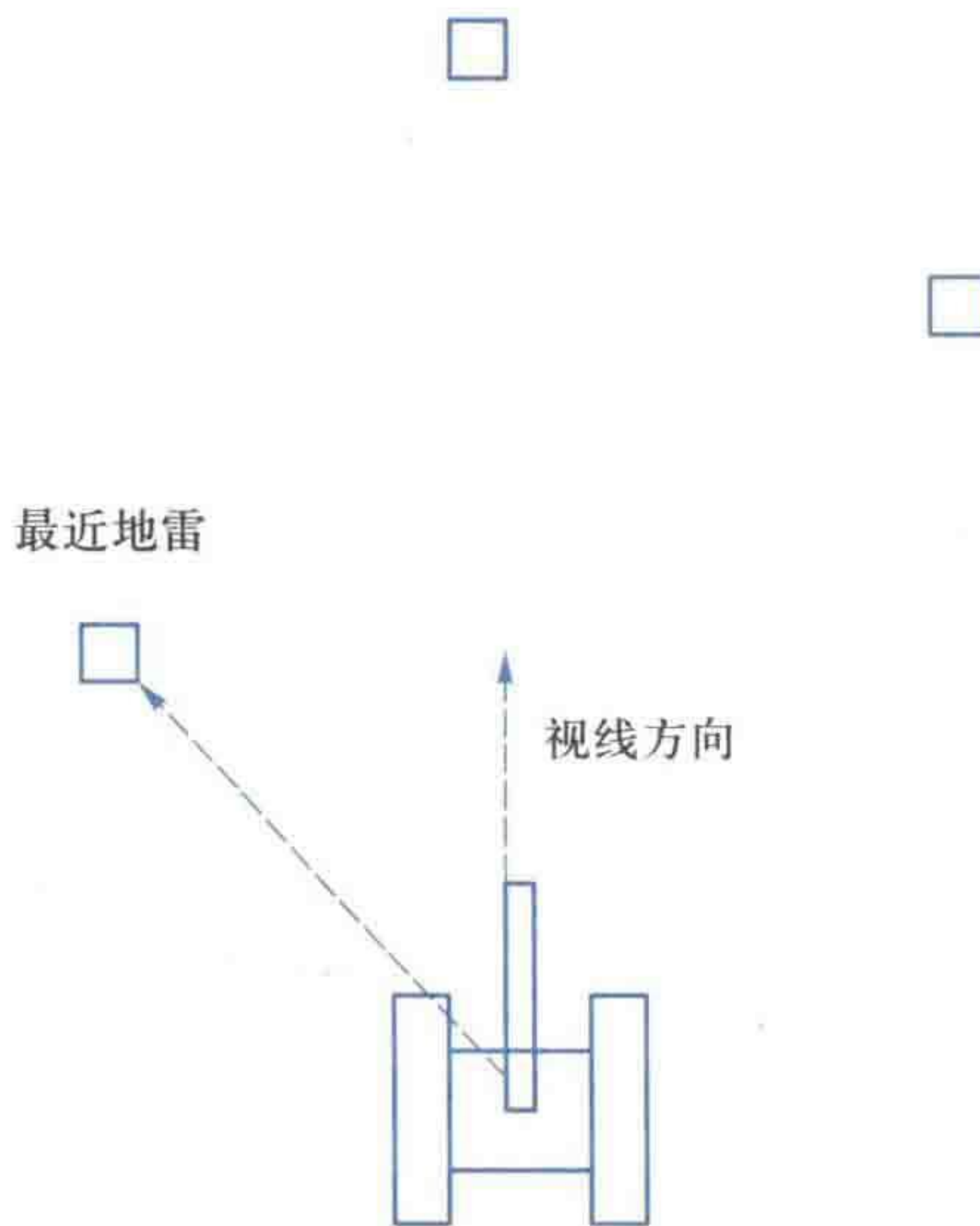


图 12.1 扫雷机的控制

选择几个(例如 10 个)神经元作为隐含层,构成一个三层 BP 神经网络,如图 12.2 所示。神经元的非线性函数取 S 型函数。

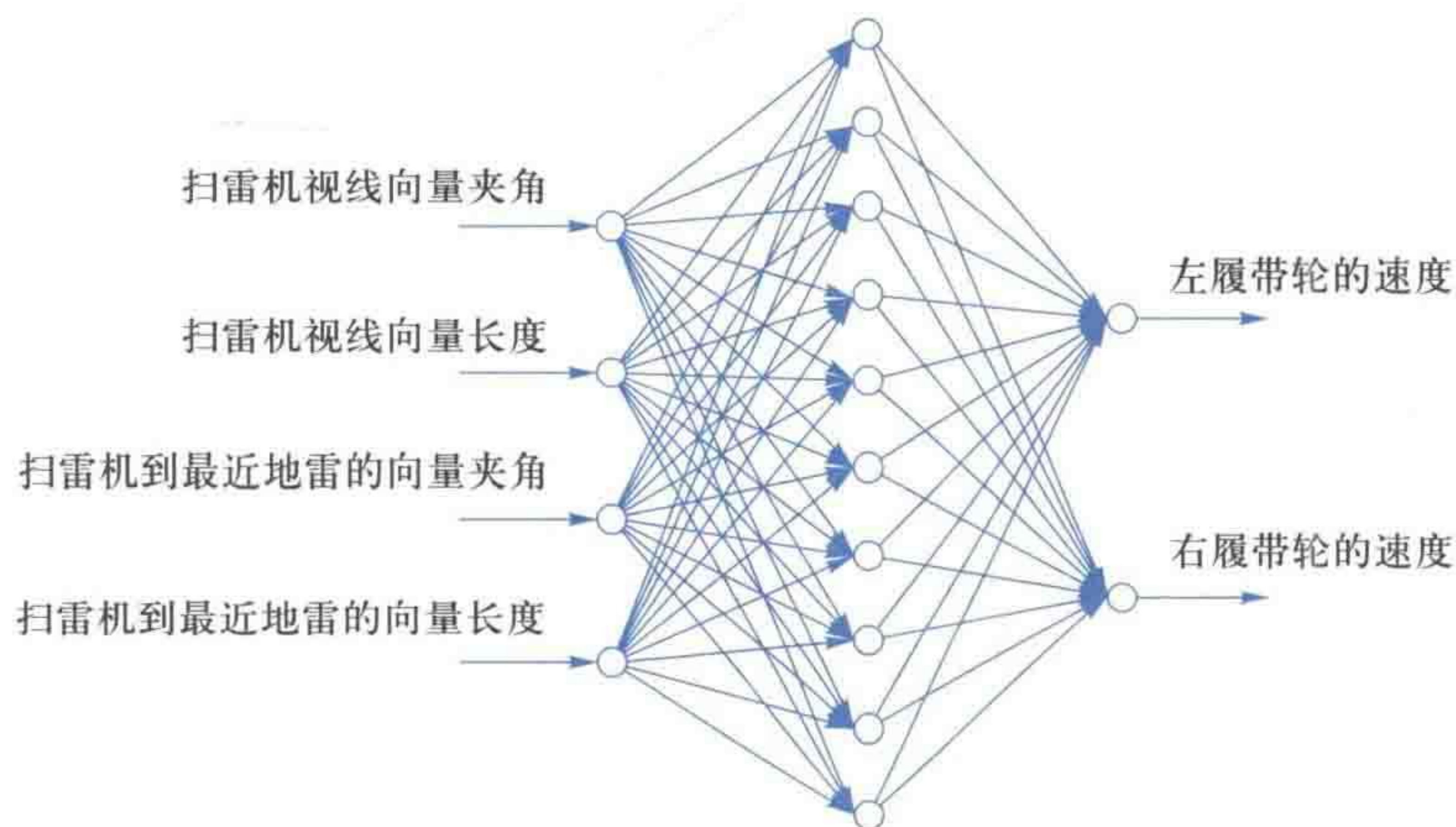


图 12.2 控制扫雷机的神经网络

可以将神经网络的所有输入进行规范化。对于扫雷机的视线向量已经是一个规范化向量,即长度等于 1,分量都在 0~1。而扫雷机到达最近地雷的向量可能很大,其中的一个分量甚至可能达到窗体宽度或者高度。如果把这个数据以它的原始状态输入到网络,将使它的影响比视线向量大很多,会使网络性能变差。因此需要把它规范化,变换到 0~1。

首先随机设置权值。然后用遗传算法进行进化。对上面得到的神经网络从左到右依次读每一层神经元的权值以及阈值,保存到一个向量中,就实现了神经网络的编码。然后采用各种方法进行选择、交叉与变异。当产生新一代,就用新个体表示的权值替换扫雷机神经网络的权值。当某个扫雷机找到了地雷就增加它的适应度。扫雷机对应的神经网络不断进化,扫雷机的智能就不断提高。

可以用神经网络解决许多游戏中的避障和搜索这两个人工智能问题。在扫雷机游戏中增加了一些要求扫雷机躲避的障碍物。障碍物的顶点坐标存放在一个缓冲区里。

设置适应性函数反映扫雷机与障碍物的碰撞情况,适应度值越高,表示扫雷机避障性能越好。如果扫雷机产生一次碰撞就降低这个扫雷机的适应度。

12.7 人工智能游戏的现状与未来

虽然多数游戏软件都具有一些人工智能的表现,例如,经典游戏“小精灵”里的魔鬼、第一人称射击游戏“半条命”里的虚拟队友以及许多介于两者之间的其他游戏角色。但现阶段游戏中的人工智能的开发,不是人工智能算法理论上的改进和实现,而是呈现给玩家一个看上去合理的行为。主要原因有以下几个方面。

① 游戏软件的实时性限制了计算量较大的人工智能算法的应用。

② 来自人工智能理论本身发展的限制。虽然专家系统、模糊逻辑、人工神经网络等方法渐渐为游戏 AI 开发者所考虑,但由于建模的复杂性,这些工具不能很好地求解游戏中的问题。

③ 游戏中的角色并不是真正的智能体。例如,由于应用环境的相似性,游戏 AI 开发者大量借鉴机器人学的成果,然而即使是比较先进的机器人,它的机械复杂性也是有限的。相比之下,骨架动画的艺术创作的复杂度几乎是无限的。这些角色的基本行为更大程度上是美术工作者创造的结果的回放,甚至是通过动作捕捉(motion capture)技术所做的人类行为的记录。现有的人工智能技术水平无法与之匹配。

④ 游戏中实现人工智能的主要目的,是为了让玩家产生真实的环境的错觉,并不是为了求解问题的性能度量最大化。例如,游戏中一个按照路径规划算法寻路的手,玩家可能认为它的寻路能力过于完美,不符合人类真实的情况。

计算机游戏不仅是人工智能的重要应用领域,而且是人工智能的高效研究平台。利用计算机游戏进行人工智能研究相对其他应用领域比较方便而又富有乐趣。例如,在机器人等实际应用系统中研究人工智能,首先要解决复杂的底层的检测、识别与控制问题。但在虚拟的计算机游戏中,角色只要查询图像数据库,就可以解决障碍识别等问题。计算机游戏中的“玩家”能够直接观察角色的行为,及时提供反馈信息,进行人机交互,发出控制指令。这些对于人工智能研究及应用都是很重要的。许多人喜欢玩游戏,也容易验证人工智能技术的有效性。特别是许多人愿意出钱,促进了游戏产业的发展和人工智能技术的研究。游戏开发中的人工智能正处于一场革命之中,必将迅速得到发展并推动整个游戏产业的飞速发展。

12.8 小 结

应用人工智能技术设计的游戏称为人工智能游戏。

适合于游戏开发的人工智能技术称为游戏人工智能。

游戏人工智能分为定性和非定性两类。用定性技术设计的角色行为是特定的、可预测的。用非定性技术设计的角色行为具有某种程度的不确定性和不可预测性。

基本的游戏人工智能技术有:搜索技术、专家系统、遗传算法、模糊逻辑、神经网络、一阶谓词逻辑、多智能体、人工生命、基于范例的推理、有限状态机、决策树、置信网络等。

游戏中的角色根据角色的属性分为玩家和非玩家;根据运动方向与朝向的相关性,分为靠转向力改变的角色和自由运动的角色;根据角色的智能性,分为智能性角色和无智能性角色。

角色的智能性具体表现在指导与运动、追逐与躲避、群聚、路径搜索、智能搜索引擎等方面。

如何指导角色行动有两种方法:预定义行为和目标导向行为。区分预定义行为和目标导向行为的依据是:角色是否唯一地选择行动。

预定义行为方法中,角色的所有行为都是动画师和程序设计人员预先设计的。

当角色的行为仅仅取决于当前外部环境的感知时,称为反应行为。

路径搜索是智能游戏软件中最基本的问题之一。 A^* 搜索算法是人工智能中的一种启发式搜索策略,用来解决最短径优化问题。 A^* 算法能保证在任何起点与终点之间找到最佳路径,但要求 CPU 做大量的计算。

智能游戏不仅是人工智能的重要应用领域,而且是人工智能的高效研究平台。人工智能技术将迅速推动整个游戏产业的飞速发展。

思考题

- 12.1 什么是人工智能游戏? 什么是游戏人工智能?
- 12.2 指导角色行动有哪两种方法? 分析两者的优点和缺点。
- 12.3 设计一个简单的智能游戏。

读一读

深度学习专家吴恩达

吴恩达(Andrew NG, 1976—)美籍华人, 计算机科学家, 全球人工智能领军人物, 硅谷的发明家、投资人和企业家, 对人工智能、深度学习、机器人学和机器学习做出了重大贡献。

1976年出生于英国伦敦, 他的父母都是来自香港的移民。1992年毕业于新加坡莱佛士书院。1997年, 他在卡内基·梅隆大学获得计算机科学、统计学和经济学三重专业学位。1996—1998年间, 在贝尔实验室进行强化学习、模型选择和特征选择方面的研究。1998年, 获得麻省理工学院硕士学位。2002年, 获得加州大学伯克利分校博士学位。他的博士论文题目是《强化学习的形成与策略研究》。

他于2002年开始在斯坦福大学任教, 担任人工智能实验室的主任。他在斯坦福大学开设的机器学习课程是校园里最受欢迎的课程之一, 几年来有1 000多名学生注册。2012年, 他和斯坦福大学的计算机科学家达芙妮·科勒共同创立了Coursera, 并担任该网站的首席执行官, 该网站为所有人提供免费在线课程, 已有数百万人报名参加Coursera课程, 使该网站成为世界领先的MOOC网站之一。

2011年至2012年, 他在谷歌工作, 创建并领导了谷歌大脑深度学习项目。2014年, 他以首席科学家的身份加入百度, 负责百度研究院的工作, 开展诸如面部识别和Melody医疗保健的AI聊天机器人等领域的研究。2017年3月, 他宣布辞去百度的职务。2017年, 他推出了在线深度学习系列课程Deeplearning.ai。

附录 A 部分习题解答

第 2 章习题解答

2.1 (1) 定义谓词: $likes(x, y)$ 为 x 喜欢 y 。 x :人; $flower1$:梅花; $flower2$:菊花。

$(\exists x)(likes(x, flower1)) \vee (\exists x)(likes(x, flower2)) \vee (\exists x)(likes(x, flower1) \wedge likes(x, flower2))$

(2) 定义谓词: $plays(z, y, x)$ 为 z 在 x 时间玩 y 。 x :下午。

$(\forall x)(plays(he, football, x))$

(3) 定义谓词: $have(x, y)$ 为 x 有 y 。 x :人。 $eat(x, y)$ 为 x 吃 y 。 x :人。

$(\forall x)(have(x, rice) \wedge eat(x, rice))$

(4) 定义谓词: $like(x, y)$ 为 x 喜欢 y 。 x :人。

$(\forall x)[like(x, play(basketball)) \rightarrow like(x, play(volleyball))]$

(5) 定义谓词: $pass(x, y)$ 为 x 通过 y 。 $study(x, y)$ 为 x 到 y 学习。 x :人。 $exam(English)$ 为外语考试。

$(\forall x)[\neg pass(x, exam(English)) \rightarrow \neg (study(x, abroad))]$

2.2 (1) $(\forall x)$ 的辖域为 $(P(x, y) \vee (\exists y)(Q(x, y) \wedge R(x, y)))$ 。 $(\exists y)$ 的辖域为 $Q(x, y) \wedge R(x, y)$ 。谓词公式中 x 为约束变元, $P(x, y)$ 中的 y 为自由变元。 $Q(x, y) \wedge R(x, y)$ 中的 y 为约束变元。

(2) $(\exists z)$ 、 $(\forall y)$ 的辖域为 $P(z, y) \vee Q(z, x)$ 。谓词公式中 z, y 为约束变元, x, u, v 为自由变元。

(3) $(\forall x)$ 的辖域为 $\neg P(x, f(x)) \vee (\exists z)(Q(x, z) \wedge \neg R(z, y))$ 。 $(\exists z)$ 的辖域为 $Q(x, z) \wedge \neg R(z, y)$ 。谓词公式中 x, z 为约束变元, y 为自由变元。

(4) $(\forall z)$ 的辖域为 $(\exists y)((\exists t)(P(z, t) \vee Q(y, t)) \wedge R(z, y))$ 。 $\exists y$ 的辖域为 $(\exists t)(P(z, t) \vee Q(y, t))$, $(\exists t)$ 的辖域为 $P(z, t) \vee Q(y, t)$ 。谓词公式中 $R(z, y)$ 的 y 为自由变元, z 为约束变元。

2.3 设对谓词指派的真值为:

$P(1, 1) = T, P(1, 2) = F, P(2, 1) = T, P(2, 2) = F$

$Q(1, 1) = F, Q(1, 2) = F, Q(2, 1) = T, Q(2, 2) = T$

上述指派就是对题中给出的谓词公式的一个解释。

当 $x=1, y=1$ 时,有 $P(1, 1) = T, Q(1, 1) = F$,则 $P(1, 1) \rightarrow Q(1, 1)$ 的真值为 F

当 $x=2, y=1$ 时, 有 $P(2,1)=T, Q(2,1)=T$, 则 $P(2,1) \rightarrow Q(2,1)$ 的真值为 T

当 $x=1, y=2$ 时, 有 $P(1,2)=F, Q(1,2)=F$, 则 $P(1,2) \rightarrow Q(1,2)$ 的真值为 T

当 $x=2, y=2$ 时, 有 $P(2,2)=F, Q(2,2)=T$, 则 $P(2,2) \rightarrow Q(2,2)$ 的真值为 T

即对个体域 D 中的所有 y 总存在 x 使 $P(x,y) \rightarrow Q(x,y)$ 的真值为 T 。所以谓词公式 $(\exists x)(\forall y)P(x,y) \rightarrow Q(x,y)$ 在该解释下的真值为 T 。

2.4 (1) $Older(x,y): x$ 比 y 大。 $Older(Zhang, Li) \rightarrow \neg Older(Li, Zhang)$

(2) $Man(x): x$ 为男; $\neg Man(x): x$ 为女; $Marry(x,y): x$ 与 y 结婚。

$Marry(甲, 乙) \rightarrow (Man(甲) \wedge \neg Man(乙)) \vee (Man(乙) \wedge \neg Man(甲))$

(3) $Honest(x): x$ 是老实人; $Lie(x): x$ 说谎。 $Honest(x) \rightarrow \neg Lie(x); Lie(Zhang) \rightarrow \neg Honest(Zhang)$

2.5 IF $x_1=0$ AND $x_2=0$ THEN $y=0$

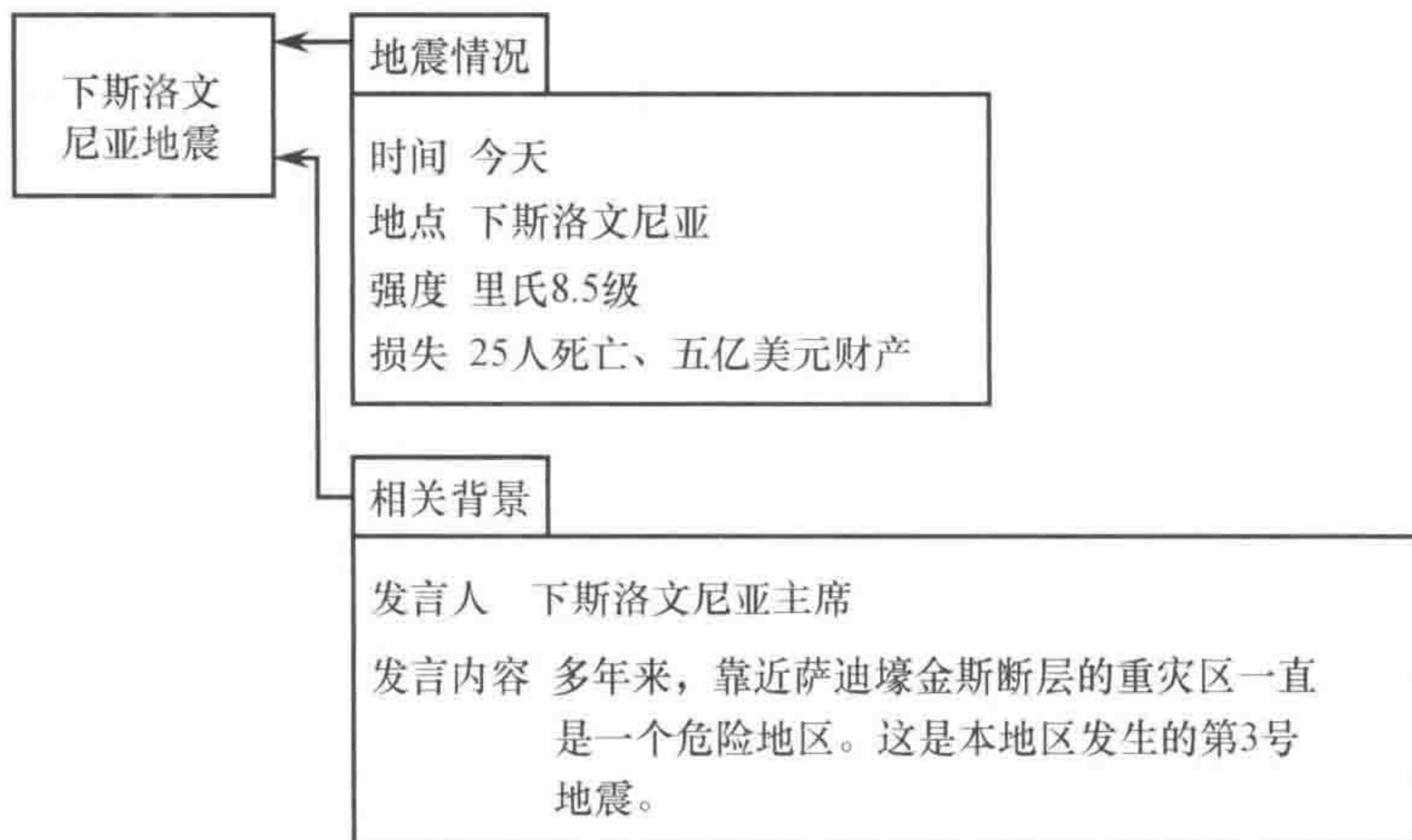
IF $x_1=0$ AND $x_2=1$ THEN $y=1$

IF $x_1=1$ AND $x_2=0$ THEN $y=1$

IF $x_1=1$ AND $x_2=1$ THEN $y=0$

2.6 IF 发烧 AND 呕吐 AND 黄疸 THEN 肝炎 0.7

2.7



第3章习题解答

3.1 因为 $A, A \rightarrow C \Rightarrow C; B, C \Rightarrow B \wedge C; B \wedge C, B \wedge C \rightarrow D \Rightarrow D; D, D \rightarrow Q \Rightarrow Q$; 所以, Q 为真。

3.2 $\neg P(x, f(x), z, g(x, z))$

3.3 $(\forall y)(\neg P(b, g(y)) \vee R(b, y, f(a)))$

3.4 (1) $\{P(z,y), Q(u,v)\}$

(2) $\{\neg P(x,y) \vee Q(x,y)\}$

(3) $\{\neg P(x,f(x)) \vee R(x,f(x)), \neg Q(y,f(y)) \vee R(y,f(y))\}$

(4) $\{\neg P(x,y) \vee R(x,y), \neg Q(u,v) \vee R(u,v)\}$

(5) $\{\neg P(x,y) \vee Q(x,y) \vee R(x,f(x,y))\}$

(6) $\{P(a,b,z,g(z),v,f(z,v)), Q(a,b,z,g(z),v,f(z,v)) \vee \neg R(a,z,f(z,v))\}$

(7) $\{\neg P(x,f(x)) \vee Q(x,g(x)), \neg P(y,f(y)) \vee \neg R(y,g(y))\}$

3.5 解:(1) $\neg P$ 与 P 归结得 NIL , 故子句集不可满足。

(2) $P \vee Q$ 与 $P \vee \neg Q$ 归结得 P ; $\neg P \vee Q$ 与 $\neg P \vee \neg Q$ 归结得 $\neg P$ 。 P 与 $\neg P$ 归结得 NIL , 故子句集不可满足。

(3) 由于存在 $R(a)$, 不可能归结出空子句, 所以该子句集是可满足的。

(4) $P(a)$ 与 $\neg P(y) \vee R(y)$ 归结得 $R(a)$; $S(a)$ 与 $\neg S(z) \vee \neg R(z)$ 归结得 $\neg R(a)$; $R(a)$ 与 $\neg R(a)$ 归结得 NIL , 故子句集不可满足。

(5) $R(b)$ 与 $\neg R(z) \vee L(a,z)$ 归结得 $L(a,b)$; $P(a)$ 与 $\neg P(x) \vee \neg Q(y) \vee \neg L(x,y)$ 归结得 $\neg Q(y) \vee \neg L(a,y)$, 与 $Q(b)$ 归结得 $\neg L(a,b)$, 再与 $L(a,b)$ 归结得 NIL , 故子句集不可满足。

3.6 (1) F_1 的子句集为 $P(a,b)$ 。 $\neg G$ 的子句集为 $\neg P(x,b)$, 归结得 NIL , 故 G 为 F_1 的逻辑结论。

(2) F_1 的子句集为 $\{P(x), Q(a) \vee Q(b)\}$; $\neg G$ 的子句集为 $\{\neg P(x) \vee \neg Q(x)\}$; $Q(a) \vee Q(b)$ 与 $\neg P(x) \vee \neg Q(x)$ 归结得 $\neg P(b)$; 再与 $P(x)$ 归结得 NIL 。

(3) F_1 的子句集为① $P(f(a))$, ② $Q(f(b))$; $\neg G$ 的子句集为③ $\neg P(f(a)) \vee \neg P(y) \vee \neg Q(y)$ 。 ①与③归结得④ $\neg P(y) \vee \neg Q(y)$; 再与②归结得⑤ $\neg P(f(b))$; 再与①归结得 NIL 。

(4) F_1 化为子句集: ① $\neg P(x) \vee \neg Q(y) \vee \neg L(x,y)$; F_2 化为子句集: ② $P(a)$, ③ $\neg R(y) \vee L(a,y)$; 对 $\neg G$ 化为子句集: ④ $R(b)$, ⑤ $Q(b)$ 。 ②与①归结得⑥ $\neg Q(y) \vee \neg L(a,y)$; ⑥与③归结得⑦ $\neg R(y) \vee \neg Q(y)$; ⑦与④归结得⑧ $\neg Q(b)$; ⑧与⑤进行归结得 NIL 。

(5) 将 $F_1, F_2, \neg G$ 化为子句集: F_1 : ① $\neg P(x) \vee Q(x)$; ② $\neg P(x) \vee R(x)$; F_2 : ③ $P(a)$; ④ $S(a)$; $\neg G$: ⑤ $\neg S(z) \vee \neg R(z)$ 。 对子句进行归结: ④与⑤归结得⑥ $\neg R(a)$; 与②进行归结得 $\neg P(a)$; 与③归结得 NIL 。

(6) 化为子句集: F_1 : 令 $y=f(z)$, ① $\neg A(z) \vee B(z) \vee D(z,f(z))$, ② $\neg A(u) \vee B(u) \vee C(f(u))$; F_2 : ③ $E(a)$, ④ $A(a)$, ⑤ $\neg D(a,v) \vee E(v)$; F_3 : ⑥ $\neg E(p) \vee \neg B(p)$; ⑦ $\neg G$: $\neg E(t) \vee \neg C(t)$ 。 ③和⑥归结得⑧ $\neg B(a)$; ⑧和②归结得⑨ $\neg A(a) \vee C(f(a))$; ⑨和④归结得⑩ $C(f(a))$; ⑩和⑦归结得⑪ $\neg E(f(a))$; ⑧和①归结得⑫ $\neg A(a) \vee D(a,f(a))$; ⑫和④归结得⑬ $D(a,f(a))$; ⑬和⑤归结得⑭ $E(f(a))$; ⑪和⑭归结得 NIL 。

3.7 定义谓词: $R(x)$ 表示 x 能够阅读; $L(x)$ 表示 x 有文化; $D(x)$ 表示 x 是海豚; $I(x)$ 表示 x

有智能。将前提和结论表示为: $(\forall x)(R(x) \rightarrow L(x)); (\forall y)(D(y) \rightarrow \neg L(y)); (\exists z)(D(z) \wedge I(z)); (\exists w)(I(w) \wedge \neg R(w))$ 。化为子句集: ① $\neg R(x) \vee L(x)$; ② $\neg D(y) \vee \neg L(y)$; ③ $D(a)$; ④ $I(a)$; ⑤ $\neg I(w) \vee R(w)$ 。⑤与④归结得⑥ $R(a)$; ⑥与①归结得⑦ $L(a)$; ⑦与②归结得⑧ $\neg D(a)$; ⑧与③归结得 NIL 。

3.8 定义谓词: $S(x, y)$ 表示 x 储蓄 y ; $M(x)$ 表示 x 是钱; $I(x)$ 表示 x 是利息; $E(x, y)$ 表示 x 获得 y 。将前提表示为谓词公式: $(\forall x)((\exists y)(S(x, y) \wedge M(y)) \rightarrow (\exists y)(I(y) \wedge E(x, y)))$; 化为子句集: ① $\neg S(x, y) \vee \neg M(y) \vee I(f(x))$; ② $\neg S(x, y) \vee \neg M(y) \vee E(x, f(x))$ 。将结论表示为谓词公式: $\neg(\exists x)I(x) \rightarrow (\forall x)(\forall y)(M(y) \rightarrow \neg S(x, y))$; 化为子句集: ③ $\neg I(z)$; ④ $S(a, b)$; ⑤ $M(b)$ 。①与④归结得⑥ $\neg M(b) \vee I(f(a))$; ⑥与⑤归结得⑦ $I(f(a))$; ⑦与③归结得 NIL 。

3.9 定义谓词: $U(x, y)$: x 使用 y ; $E(u, v)$: u 得到 v ; $I(z)$: z 是 Internet; $F(u)$: u 是信息。把已知前提表示成谓词公式: $F: (\forall x)((\exists y)(U(x, y) \wedge I(y)) \rightarrow (\exists u)(F(u) \wedge E(x, u)))$ 。

化为子句集: ① $\neg U(x, y) \vee \neg I(y) \vee F(f(x))$; ② $\neg U(x, y) \vee \neg I(y) \vee E(x, f(x))$ 。把待求证的结论表示成谓词公式并否定得 $\neg G: \neg(\neg(\exists u)F(u) \rightarrow (\forall x)(\forall y)(I(y) \rightarrow \neg U(x, y)))$ 。化为子句集: ③ $\neg F(u)$; ④ $I(b)$; ⑤ $U(a, b)$ 。①与④归结得⑥ $\neg U(x, b) \vee F(f(x))$; ⑤与⑥归结得⑦ $F(f(a))$; ⑦与③归结得⑧ NIL 。

3.10 把前提表示为谓词公式: $F_1: \forall x \forall y \forall z (F(x, y) \wedge F(y, z) \rightarrow G(x, z)); F_2: F(Lao, Da); F_3: F(Da, Xiao)$ 。化为子句集: ① $\neg F(x, y) \vee \neg F(y, z) \vee G(x, z)$; ② $F(Lao, Da)$; ③ $F(Da, Xiao)$ 。

设求证的公式为 $G: \exists x \exists y G(x, y)$; 把其否定并与答案谓词析取, 化为子句集得④ $\neg G(u, v) \vee ANSWER(u, v)$ 。①, ②归结得⑤ $\neg F(Da, z) \vee G(Lao, z)$; ③, ⑤归结得⑥ $G(Lao, Xiao)$; ④, ⑥归结得⑦ $ANSWER(Lao, Xiao)$ 。所以, 老李是小李的祖父。

3.11 定义谓词: $AT(y, x)$: y 在 x 处。把已知前提表示成谓词公式: $F_1: (\forall x)(AT(Zhang, x) \rightarrow AT(Li, x)); F_2: AT(Zhang, School)$ 。把待求解的问题表示成谓词公式, 把它否定并与答案谓词析取, 得 $G: \neg(\exists x)AT(Li, x) \vee ANSWER(x)$ 。化为子句集: ① $\neg AT(Zhang, x) \vee AT(Li, x)$; ② $AT(Zhang, School)$; ③ $\neg AT(Li, x) \vee ANSWER(x)$ 。①与②归结得④ $AT(Li, School)$; ④与③归结得⑤ $ANSWER(School)$ 。由 $ANSWER(School)$ 得知小李在学校。

3.12 定义谓词: $A(x)$: x 是 ALPINE 的一个成员; $B(x)$: x 是滑雪运动员; $C(x)$: x 是登山运动员; $L(x, y)$: x 喜欢 y 。

将前提表示成谓词公式: $A(TONY), A(MIKE), A(JOHN); (\forall x)(A(x) \rightarrow [B(x) \wedge \neg C(x)] \vee [\neg B(x) \wedge C(x)]); (\forall x)(C(x) \rightarrow \neg L(x, Rain)); (\forall x)(\neg L(x, Snow) \rightarrow \neg B(x)); (\forall x)(L(TONY, x) \rightarrow \neg L(MIKE, x)); (\forall x)(\neg L(TONY, x) \rightarrow L(MIKE, x)); L(TONY, Rain); L(TONY, Snow)$ 。化为子句集: ① $A(TONY)$; ② $A(MIKE)$; ③ $A(JOHN)$; ④ $\neg A(x) \vee B(x) \vee \neg B(x)$;

⑤ $\neg A(x) \vee B(x) \vee C(x)$; ⑥ $\neg A(x) \vee \neg B(x) \vee \neg C(x)$; ⑦ $\neg A(x) \vee \neg C(x) \vee C(x)$; ⑧ $\neg C(x) \vee \neg L(x, \text{Rain})$; ⑨ $L(x, \text{Rain}) \vee \neg B(x)$; ⑩ $\neg L(\text{TONY}, x) \vee \neg L(\text{MIKE}, x)$; ⑪ $L(\text{TONY}, x) \vee L(\text{MIKE}, x)$; ⑫ $L(\text{TONY}, \text{Rain})$; ⑬ $L(\text{TONY}, \text{Snow})$

将待求解的问题表示成谓词公式, 然后否定并与答案谓词析取: $\neg (A(x) \wedge C(x) \wedge \neg B(x)) \vee \text{ANSWER}(x)$ 。化为子句集: ⑭ $\neg A(x) \vee \neg C(x) \vee B(x) \vee \text{ANSWER}(x)$ 。⑩与⑫归结得⑮ $\neg L(\text{MIKE}, \text{Rain})$; ⑮与⑨归结得⑯ $\neg B(\text{MIKE})$; ⑯与⑤归结得⑰ $\neg A(\text{MIKE}) \vee C(\text{MIKE})$; ⑰与②归结得⑱ $C(\text{MIKE})$; ⑱与⑭归结得⑲ $\neg A(\text{MIKE}) \vee B(\text{MIKE}) \vee \text{ANSWER}(\text{MIKE})$; ⑲与②归结得⑳ $B(\text{MIKE}) \vee \text{ANSWER}(\text{MIKE})$; ㉑与⑯归结得 $\text{ANSWER}(\text{MIKE})$ 。因此, MIKE 是 ALPINE 俱乐部的一个成员, 他是一个登山运动员, 但不是一个滑雪运动员。

第 4 章习题解答

$$4.1 \quad CF(E_2) = 0.6 \times \max\{0, CF(E_1)\} = 0.6 \times \max\{0, 0.5\} = 0.3$$

$$CF(E_4) = 0.8 \times \max\{0, CF(E_2 \text{ AND } E_3)\} \\ = 0.8 \times \max\{0, \min\{CF(E_2), CF(E_3)\}\} = 0.24$$

$$CF_3(H) = 0.7 \times \max\{0, CF(E_4)\} = 0.168$$

$$CF_4(H) = 0.9 \times \max\{0, CF(E_5)\} = 0.36$$

又因为 $CF_3(H) > 0, CF_4(H) > 0$, 故

$$CF(H) = CF_3(H) + CF_4(H) - CF_3(H)CF_4(H) = 0.47$$

$$4.2 \quad CF_1(H_1) = 0.7 \times \max\{0, CF(E_1)\} = 0.35$$

$$CF_2(H_1) = 0.6 \times \max\{0, CF(E_2)\} = 0.3$$

$$CF_3(H_1) = 0.4 \times \max\{0, CF(E_3)\} = 0.2$$

$$CF_{1,2}(H_1) = CF_1(H_1) + CF_2(H_1) - CF_1(H_1)CF_2(H_1) = 0.545$$

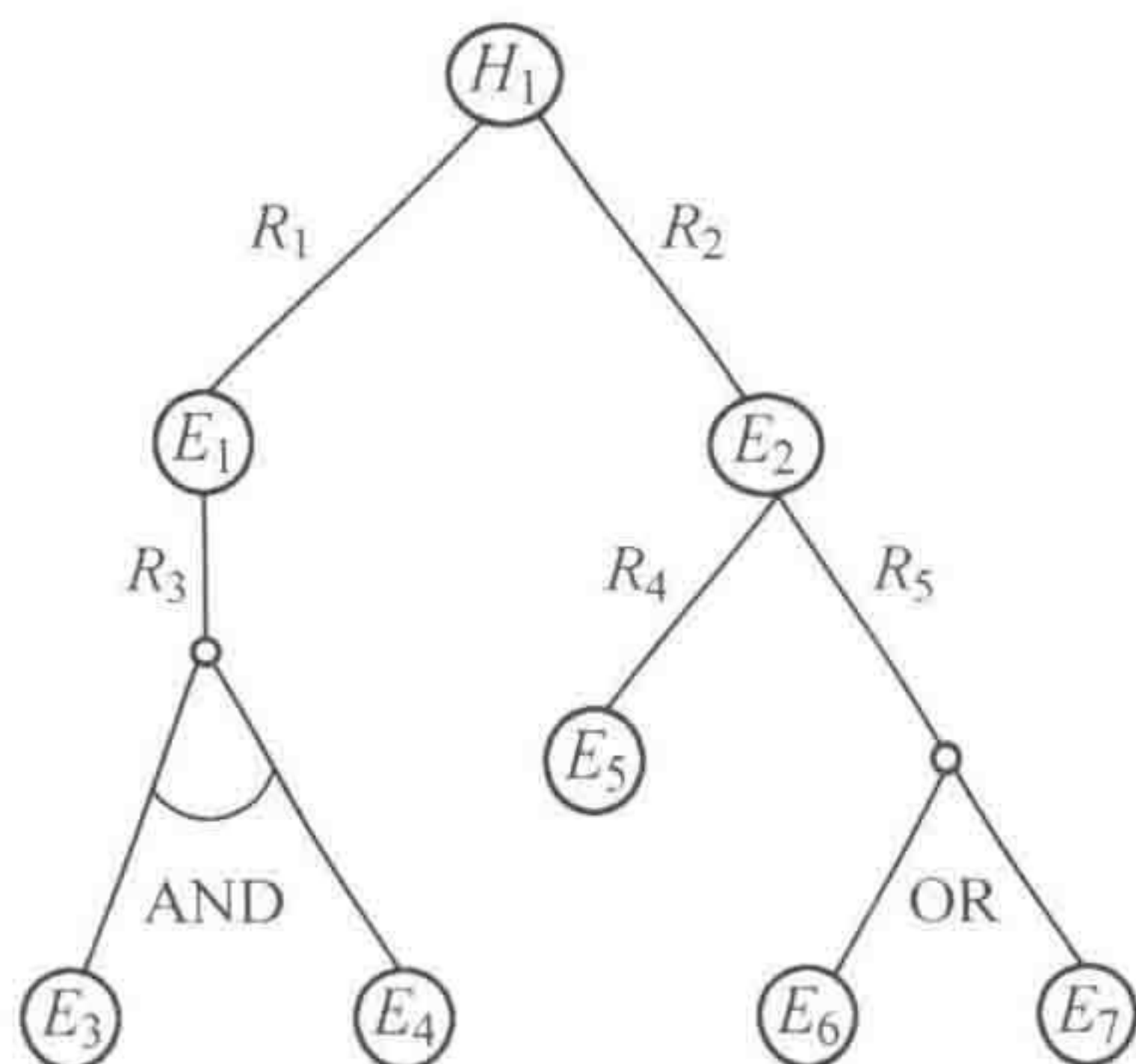
$$\text{同理 } CF(H_1) = CF_{1,2,3}(H_1) = 0.2 + 0.545 - 0.545 \times 0.2 = 0.636$$

$$\text{则 } CF_4(H_2) = 0.2 \times \max\{0, CF(H_1 \text{ AND } E_4)\}$$

$$= 0.2 \times \max\{0, \min\{CF(H_1), CF(E_4)\}\} = 0.1$$

$$CF(H_2) = CF_4(H_2) + CF_0(H_2) - CF_4(H_2)CF_0(H_2) = 0.37$$

4.3



$$CF(E_3 \text{ AND } E_4) = \min\{CF(E_3), CF(E_4)\} = 0.8$$

$$\text{所以 } CF(E_1) = 0.9 \times \max\{0, CF(E_3 \text{ AND } E_4)\} = 0.72$$

$$CF(E_6 \text{ OR } E_7) = \max\{CF(E_6), CF(E_7)\} = 0.5$$

$$\text{所以 } CF_5(E_2) = (-0.3) \times \max\{0, CF(E_6 \text{ OR } E_7)\} = -0.15$$

$$CF_4(E_2) = 0.7 \times \max\{0, CF(E_5)\} = 0.56$$

$$CF(E_2) = \frac{CF_5(E_2) + CF_4(E_2)}{1 - \min\{|CF_5(E_2)|, |CF_4(E_2)|\}} = \frac{0.41}{0.85} = 0.48$$

$$CF_1(H_1) = 0.8 \times \max\{0, CF(E_1)\} = 0.576$$

$$CF_2(H_1) = 0.9 \times \max\{0, CF(E_2)\} = 0.432$$

$$\text{所以, } CF(H_1) = CF_1(H_1) + CF_2(H_1) - CF_1(H_1) \times CF_2(H_1) = 0.759$$

$$\begin{aligned} 4.4 \quad \text{因为 } K &= \sum_{X \cap Y \neq \emptyset} M_1(X) M_2(Y) \\ &= M_1(\{b, c, d\}) M_2(\{a, b\}) + M_1(\{b, c, d\}) M_2(\{a, b, c, d\}) \\ &\quad + M_1(\{a, b, c, d\}) M_2(\{a, b\}) + M_1(\{a, b, c, d\}) M_2(\{a, b, c, d\}) \\ &= 0.7 \times 0.6 + 0.7 \times 0.4 + 0.3 \times 0.6 + 0.3 \times 0.4 = 1 \end{aligned}$$

$$\text{所以 } M(A) = K^{-1} \times \sum_{X \cap Y = A} M_1(X) M_2(Y) = \sum_{X \cap Y = A} M_1(X) M_2(Y)$$

$$\text{所以 } M(b) = M_1(\{b, c, d\}) \times M_2(\{a, b\}) = 0.7 \times 0.6 = 0.42$$

$$M(b, c, d) = M_1(\{b, c, d\}) \times M_2(\{a, b, c, d\}) = 0.7 \times 0.4 = 0.28$$

$$M(a, b) = M_1(\{a, b, c, d\}) \times M_2(\{a, b\}) = 0.3 \times 0.6 = 0.18$$

$$M(a, b, c, d) = M_1(\{a, b, c, d\}) \times M_2(\{a, b, c, d\}) = 0.3 \times 0.4 = 0.12$$

M 的其余基本概率分配函数为 0。

4.5

$$A \cap B = 0.5/x_1 + 0.65/x_2 + 0.8/x_3 + 0.9/x_4 + 0.7/x_5$$

$$A \cup B = 0.85/x_1 + 0.7/x_2 + 0.9/x_3 + 0.98/x_4 + 0.77/x_5$$

$$\neg A = 0.15/x_1 + 0.3/x_2 + 0.1/x_3 + 0.1/x_4 + 0.3/x_5$$

4.6

$$A \circ B = \begin{bmatrix} 0.7 & 0.4 \\ 0.7 & 0.4 \\ 0.5 & 0.4 \end{bmatrix}$$

4.7

$$R_1 \circ R_2 = \begin{bmatrix} 0.4 & 0.8 \\ 0.4 & 0.9 \\ 0.7 & 0.5 \\ 0.7 & 0.6 \end{bmatrix}$$

4.8

$$R_1 \circ R_2 = \begin{bmatrix} 0.6 & 0.6 & 0.0 \\ 0.3 & 0.3 & 0.1 \\ 0.0 & 0.5 & 0.1 \end{bmatrix}; R_1 \circ R_3 = \begin{bmatrix} 1.0 & 0.0 & 0.7 \\ 0.3 & 0.2 & 0.3 \\ 0.7 & 0.5 & 1.0 \end{bmatrix}; R_1 \circ R_2 \circ R_3 = \begin{bmatrix} 0.6 & 0.6 & 0.6 \\ 0.3 & 0.3 & 0.3 \\ 0.1 & 0.5 & 0.1 \end{bmatrix}$$

4.9

$$S = \begin{bmatrix} 0.8 & 0.3 & 0.2 \\ 0.3 & 0.8 & 0.7 \\ 0.3 & 0.7 & 0.8 \end{bmatrix}$$

第5章习题解答

5.1 以变量 m 和 c 表示修道士和野人在左岸和船上的实际人数, 变量 b 表示船是否在左岸, $b=1$ 表示在, $b=0$ 表示不在。问题状态用三元组 (m, c, b) 表示, 则问题求解的任务为: $(3, 3, 1) \rightarrow (0, 0, 0)$ 。在这个问题上, 状态空间可能的状态总数为 $4 \times 4 \times 2 = 32$, 但由于遵守约束: $m+c \leq 2, m \geq c$, 只有 20 个是合法的。例如, $(1, 0, 1), (1, 2, 1), (2, 3, 1)$ 等是不合法的。由于存在不合法的状态, 导致某些合法的状态不可达, 例如, $(0, 0, 1), (0, 3, 1)$, 所以这个问题只有 16 个可达的合法状态。渡河问题的操作算子可以定义两类: $L(m, c)$ 、 $R(m, c)$ 分别表示船从左岸划到右岸, 和船从右岸划到左岸。由于 m, c 取值的可能组合只有 5 个: 10, 20, 11, 01, 02, 所以总共有 10 个操作算子。可以画出渡河问题的状态空间的有向图。

附录 B 实验指导书

实验一 产生式系统实验

一、实验目的

熟悉产生式表示法,掌握产生式系统的运行机制,以及基于规则推理的基本方法。

二、实验内容

设计并编程实现一个小型产生式系统(如分类、诊断等类型)。编程语言不限,如 Python 等。

三、实验要求

1. 具体应用领域自选,具体系统名称自定。
2. 用产生式规则作为知识表示,利用产生式系统实验程序,建立知识库,分别运行正、反向推理。

四、实验报告要求

1. 编辑知识库,通过输入规则或修改规则等,建立规则库。
2. 建立事实库(综合数据库),输入多条事实或结论。
3. 运行推理,包括正向推理和反向推理,给出相应的推理过程、事实区和规则区。
4. 总结实验心得体会。

实验二 洗衣机模糊推理系统实验

一、实验目的

理解模糊逻辑推理的原理及特点,掌握模糊推理方法。

二、实验内容

采用 Python 或 Matlab 7.0 的 Fuzzy Logic Tool 设计洗衣机洗涤时间的模糊推理。

三、实验要求

已知人工操作经验为

“污泥越多,油脂越多,洗涤时间越长”;

“污泥适中,油脂适中,洗涤时间适中”;

“污泥越少,油脂越少,洗涤时间越短”。

模糊推理规则如表 B.1 所示。

表 B.1 洗衣机的模糊推理规则表

污泥量 x	油脂量 y	洗涤时间 z
SD	NG	VS
SD	MG	M
SD	LG	L
MD	NG	S
MD	MG	M
MD	LG	L
LD	NG	M
LD	MG	L
LD	LG	VL

其中 SD(污泥少)、MD(污泥中)、LD(污泥多)、NG(油脂少)、MG(油脂中)、LG(油脂多)、VS(洗涤时间很短)、S(洗涤时间短)、M(洗涤时间中等)、L(洗涤时间长)、VL(洗涤时间很长)。

- (1) 假设污泥、油脂、洗涤时间的论域分别为 $[0,100]$ 、 $[0,100]$ 和 $[0,120]$,设计相应的模糊推理系统,给出输入、输出语言变量的隶属函数图,模糊推理规则表。
- (2) 假定当前传感器测得的信息为 x_0 (污泥)=60, y_0 (油脂)=70,采用模糊决策,给出模糊推理结果。

四、实验报告要求

1. 按照实验要求,给出相应结果。
2. 分析隶属度、模糊关系和模糊规则的相互关系。
3. 总结实验心得体会。

实验三 汽车控制模糊推理系统实验

一、实验目的

理解模糊逻辑推理的原理及特点,熟练应用模糊推理方法。

二、实验内容

采用 Matlab 7.0 的 Fuzzy Logic Tool 设计汽车控制模糊推理系统。

三、实验要求

假设两辆汽车均为理想状态,即 $\frac{Y(s)}{U(s)} = \frac{4}{s^2 + 2 \times 0.7 \times 2s + 4}$, Y 为速度, U 为油门控制输入。

(1) 设计模糊推理系统控制 2 号汽车由静止启动,追赶 200 m 外时速 90 km 的 1 号汽车并与其保持 30 m 的距离。

(2) 在 25 时刻 1 号汽车速度改为时速 110 km 时,仍与其保持 30 m 距离。

(3) 在 35 时刻 1 号汽车速度改为时速 70 km 时,仍与其保持 30 m 距离。

模糊推理规则中 u 由 θ 、 r 控制,如表 B.2 所示,其中 $r = \sqrt{e^2 + \dot{e}^2}$, $\theta = \tan^{-1} \frac{\dot{e}}{e}$, r 、 θ 和油门控制 u 的论域分别为 $[0, 1]$ 、 $[-3, 3]$ 和 $[-1, 1]$ 。

表 B.2 u 的模糊推理规则表

θ	r		
	NB	ZE	PB
PB	ZE	NM	NB
PM	ZE	PM	PB
ZE	ZE	PM	PB
NM	ZE	NM	NB
NB	ZE	NM	NB

r 的隶属函数如图 B.1 所示。

(1) 如图 B.2 所示,设计两输入一输出的模糊推理系统作为 2 号汽车的模糊控制器,其中输入为误差 e 和误差的变化 $\dot{e}$,输出为 1 号汽车的油门控制 u ,采用中位数法进行模糊决策,给出输入、输出语言变量的隶属函数图,模糊推理规则表,模糊推理结果。

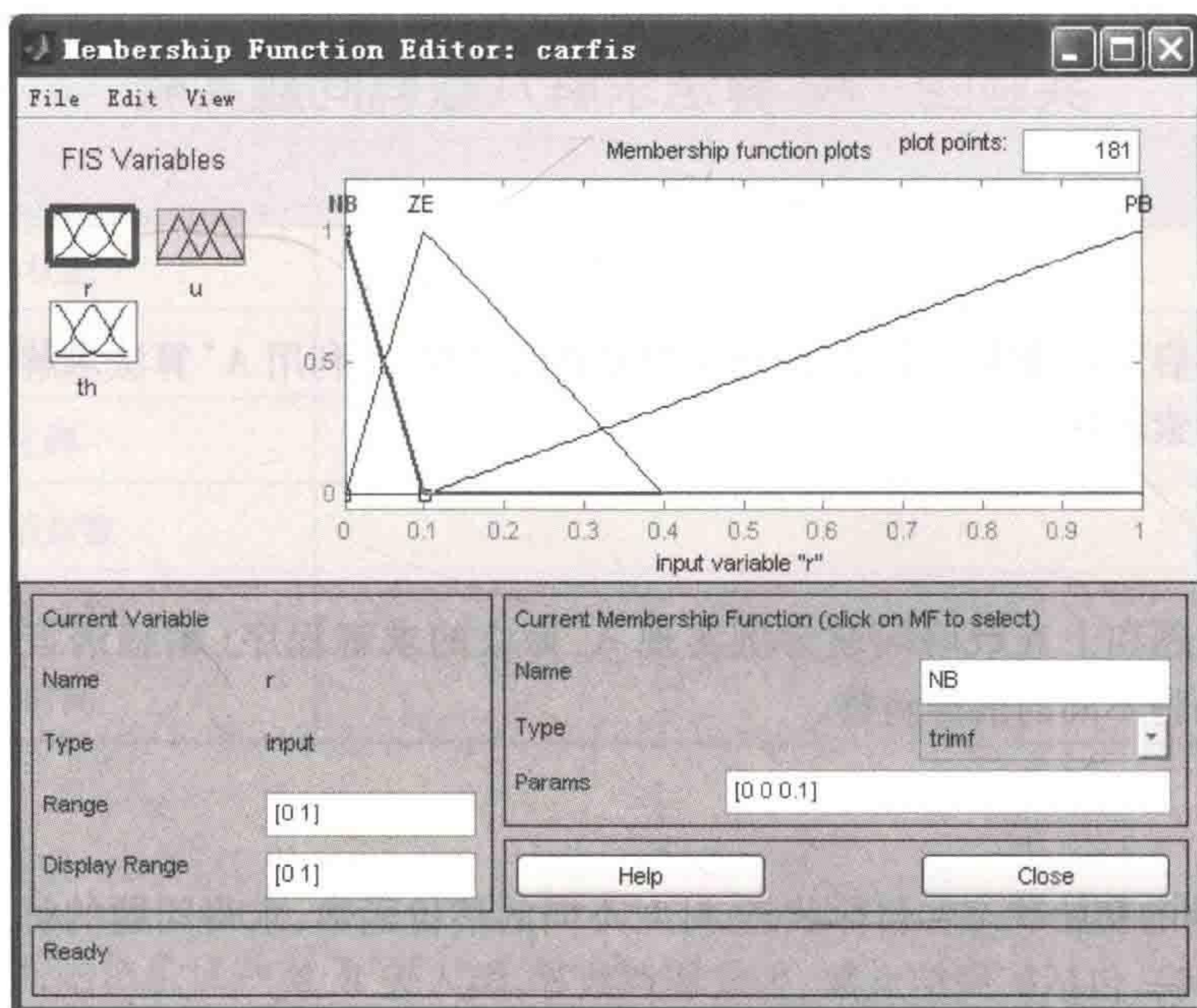
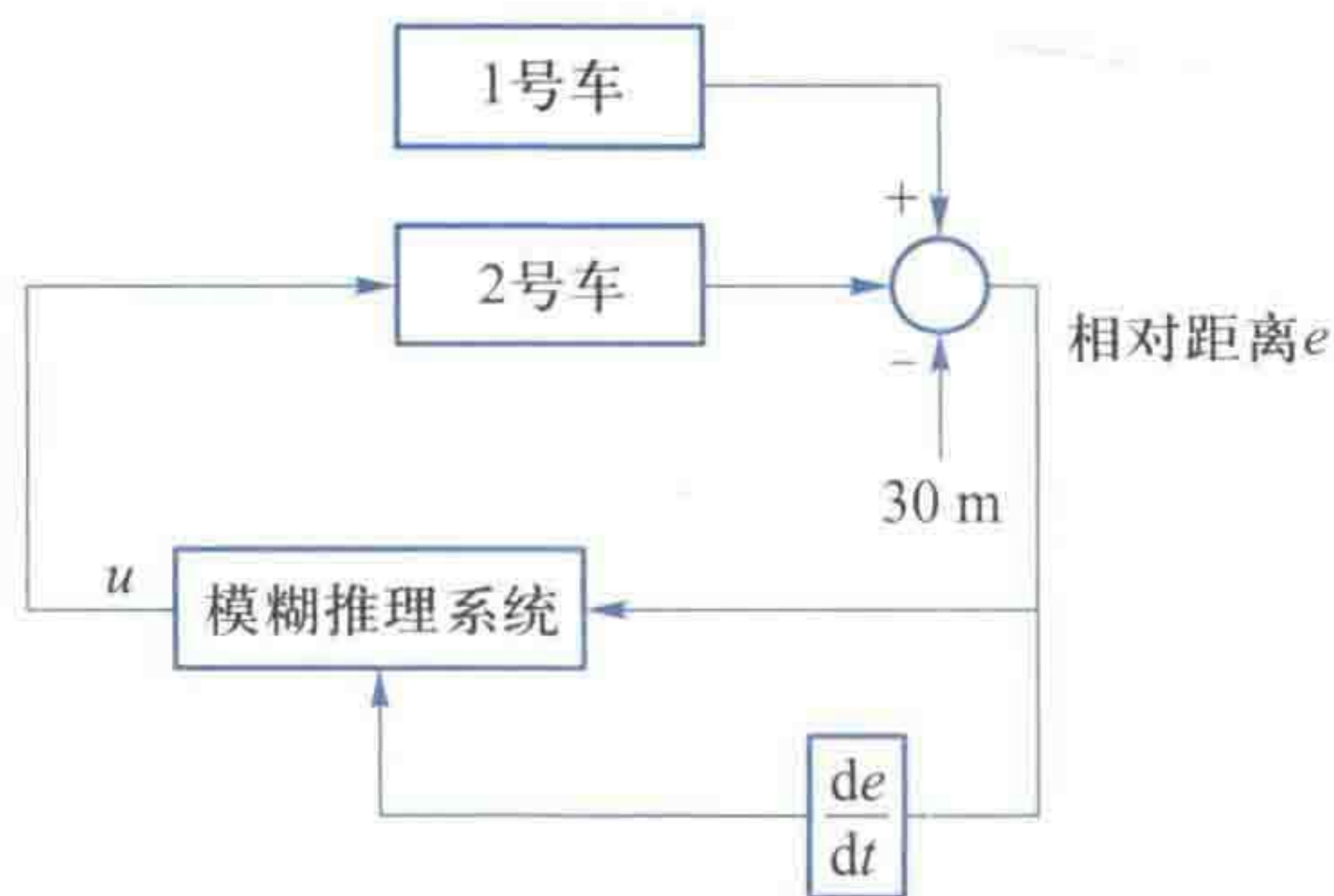
图 B.1 r 的隶属函数图

图 B.2 两车追赶的模糊控制系统框图

(2) 用 SIMULINK 仿真两车追赶的模糊控制系统, 给出目标车(1 号汽车)的速度曲线图, 以及追赶车(2 号汽车)的速度曲线图和与目标车(1 号汽车)相对距离变化图。

四、实验报告要求

1. 按照实验要求, 给出相应结果。
2. 分析隶属度、模糊关系和模糊规则的相互关系。
3. 总结实验心得体会。

实验四 A* 算法求解八数码问题实验

一、实验目的

熟悉和掌握启发式搜索的定义、估价函数和算法过程,并利用 A* 算法求解 N 数码难题,理解求解流程和搜索顺序。

二、实验内容

以八数码问题和十五数码问题为例实现 A* 算法的求解程序(编程语言不限,如 Python 等),要求设计两种不同的估价函数。

三、实验要求

1. 设置相同的初始状态和目标状态,针对不同的估价函数,求得问题的解,比较它们对搜索算法性能的影响,包括扩展结点数、生成结点数等,填入表 B.3。

表 B.3 不同启发函数 $h(n)$ 求解八数码问题的结果比较

	启发函数 $h(n)$	
	不在位数	
初始状态		
目标状态	123804765	123804765
最优解		
扩展结点数		
生成结点数		
运行时间		

2. 设置与上述 1 相同的初始状态和目标状态,用宽度优先搜索算法(即令估计代价 $h(n)=0$ 的 A* 算法)求得问题的解,以及搜索过程中的扩展结点数、生成结点数,填入表 B.3。

3. 实现 A* 算法求解十五数码问题的程序,设计两种不同的估价函数,然后重复上述 1 和 2 的实验内容,把结果填入表 B.4。

表 B.4 不同启发函数 $h(n)$ 求解十五数码问题的结果比较

	启发函数 $h(n)$	
	不在位数	
初始状态		
目标状态		
最优解		
扩展结点数		
生成结点数		
运行时间		

四、实验报告要求

1. 分析不同的估价函数对 A* 算法性能的影响。
2. 根据宽度优先搜索算法和 A* 算法求解八、十五数码问题的结果,分析启发式搜索的特点。
3. 画出 A* 算法求解 N 数码问题的流程图。
4. 总结实验心得体会。

实验五 A* 算法求解迷宫寻路问题实验

一、实验目的

熟悉和掌握 A* 算法实现迷宫寻路功能,要求掌握启发式函数的编写以及各类启发式函数效果的比较。

二、实验内容

寻路问题常见于各类游戏中角色寻路、三维虚拟场景中运动目标的路径规划、机器人寻路等多个应用领域。迷宫寻路问题是在以方格表示的地图场景中,对于给定的起点、终点和障碍物(墙),如何找到一条从起点开始避开障碍物到达终点的最短路径。

假设在一个 $n \times m$ 的迷宫里,入口坐标和出口坐标分别为(1,1)和(5,5),每一个坐标点有两种可能:0 或 1,其中 0 表示该位置允许通过,1 表示该位置不允许通过。

如地图:

0 0 0 0 0

1 0 1 0 1

0 0 1 1 1

0 1 0 0 0

0 0 0 1 0

最短路径应该是

A B 0 0 0

1 C 1 0 1

E D 1 1 1

F 1 J K L

G H I 1 M

即(1,1)-(1,2)-(2,2)-(3,2)-(3,1)-(4,1)-(5,1)-(5,2)-(5,3)-(4,3)-(4,4)-(4,5)-(5,5)

以寻路问题为例实现 A* 算法的求解程序(编程语言不限,如 Python 等),要求设计两种不同的估价函数。

三、实验要求

1. 画出用 A* 算法求解迷宫最短路径的流程图。
2. 设置不同的地图,以及不同的初始状态和目标状态,记录 A* 算法的求解结果,包括最短路径、扩展结点数、生成结点数和算法运行时间。
3. 对于相同的初始状态和目标状态,设计不同的启发式函数,比较不同启发式函数对迷宫寻路速度的提升效果,包括扩展结点数、生成结点数和算法运行时间。

四、实验报告要求

1. 画出 A* 算法求解迷宫最短路径问题的流程图。
2. 分析不同启发式函数 $h(n)$ 对迷宫寻路求解的速度提升效果。
3. 分析 A* 算法求解不同规模迷宫最短路径问题的性能。
4. 总结实验心得体会。

实验六 遗传算法求函数最大值实验

一、实验目的

熟悉和掌握遗传算法的原理、流程和编码策略,并利用遗传算法求解函数优化问题,理解求解流程并测试主要参数对结果的影响。

二、实验内容

采用 Python 或 Matlab 7. X 的遗传算法工具箱求解函数最大值的程序。

三、实验要求

1. 用遗传算法求解下列函数的最大值,设定求解精度到 15 位小数。

$$f(x,y)=\frac{6.452(x+0.125y)[\cos(x)-\cos(2y)]^2}{\sqrt{0.8+(x-4.2)^2+2(y-7)^2}}+3.226y$$
$$x\in[0,10],\quad y\in[0,10]$$

(1) 设计及选择上述问题的编码、选择、交叉、变异以及控制参数等,填入表 B. 5。

(2) 使用相同的初始种群,设置不同的种群规模,如 5、20 和 100,初始种群的个体取值范围为[0,10],其他参数同表 B. 5,然后求得相应的最佳个体最佳适应度、平均适应度,填入表 B. 6,分析种群规模对算法性能的影响。

(3) 设置种群规模为 20,初始种群的个体取值范围为[0,10],选择不同的选择、交叉和变异,其他参数同表 B. 5,然后独立运行算法 10 次,完成表 B. 7,并分析比较采用不同的选择、交叉和变异策略的算法运行结果。

表 B. 5 遗传算法参数的选择

编码	编码方式(population type)	
种群参数	种群规模(population size)	
	初始种群的个体取值范围(initial range)	
选择操作	个体选择概率分配策略(fitness scaling)	
	个体选择方法(selection function)	
最佳个体保存	优良个体保存数量(elite count)	
交叉操作	交叉概率(crossover fraction)	
	交叉方式(crossover function)	
变异操作	变异方式(mutation function)	
	变异概率(mutation fraction)	
停止参数	最大迭代步数(generations)	
	最大运行时间限制(time limit)	
	最小适应度限制(fitness limit)	

表 B.6 不同的种群规模的 GA 运行结果

种群规模	最佳适应度	平均适应度	最佳个体
5			
20			
100			

表 B.7 不同的选择、交叉和变异策略的算法运行结果

遗传算法参数设置(gaoptimset)			1	2	3	4
选择操作	个体选择概率分配 FitnessScalingFcn	Rank(排序) @ fitscalingrank	√	√		√
		Proportional(比率) @ fitscalingprop			√	
	个体选择 SelectionFcn	Roulette(轮盘赌选择) @_ selectionroulette	√	√		√
		Tournament(锦标赛选择) @_ selectiontournament			√	
交叉操作 CrossoverFcn	单点交叉 @ crossoveringlepoint		√		√	√
	两点交叉 @ crossovertwopoint			√		
变异操作 MutationFcn	Uniform(均匀变异)@_ mutationuniform		√	√	√	
	Gaussian(高斯变异)@_ mutationgaussian					√
最好适应度						
最差适应度						
平均适应度						

2. 用遗传算法求解下面 Rastrigin 函数的最小值, 设定求解精度到 15 位小数。

$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos 2\pi x_1 + \cos 2\pi x_2)$$

(1) 设计上述问题的编码、选择、交叉、变异以及控制参数等, 填入表 B. 8, 并画出最佳适应度和最佳个体图。

表 B.8 遗传算法参数的选择

编码	编码方式 (population type)	
种群参数	种群规模 (population size)	
	初始种群的个体取值范围 (initial range)	
选择操作	个体选择概率分配策略 (对应 fitness scaling)	
	个体选择方法 (selection function)	
最佳个体保存	优良个体保存数量 (elite count)	
交叉操作	交叉概率 (crossover fraction)	
	交叉方式 (crossover function)	
变异操作	变异方式 (mutation function)	
	变异概率 (mutation fraction)	
停止参数	最大迭代步数 (generations)	
	最大运行时间限制 (time limit)	
	最小适应度限制 (fitness limit)	

(2) 设置种群的不同初始范围,例如 $[1, 1.1]$ 、 $[1, 100]$ 和 $[1, 2]$,画出相应的最佳适应度图,比较分析初始范围及种群多样性对遗传算法性能的影响。

(3) 设置不同的交叉概率 (crossover fraction = 0, 0.8, 1),画出无变异的交叉 (crossover fraction = 1)、无交叉的变异 (crossover fraction = 0) 以及交叉概率为 0.8 时最佳适应度值 (best fitness) 图,分析交叉和变异操作对算法性能的影响。

四、实验报告要求

- 1. 画出遗传算法的算法流程图。
- 2. 根据实验内容,给出相应结果以及结果分析。
- 3. 总结遗传算法的特点,并说明适应度函数在遗传算法中的作用。
- 4. 总结实验心得体会。

实验七 遗传算法求 TSP 问题实验

一、实验目的

熟悉和掌握遗传算法的原理、流程和编码策略,理解求解 TSP 问题的流程并测试主要参数对结果的影响。

二、实验内容

用遗传算法求解不同规模的 TSP 问题(编程语言不限,如 Python 等)。

三、实验要求

1. 用遗传算法求解不同规模(如 10 个城市、20 个城市、100 个城市)的 TSP 问题,把结果填入表 B.9。

表 B.9 遗传算法求解不同规模的 TSP 问题的结果

城市规模	最好适应度	最差适应度	平均适应度	平均运行时间
10				
20				
100				

2. 对于同一个 TSP 问题(如 10 个城市),设置不同的种群规模、交叉概率和变异概率,把结果填入表 B.10。

表 B.10 不同的种群规模、交叉概率和变异概率的求解结果

种群规模	交叉概率	变异概率	最好适应度	最差适应度	平均适应度	平均运行时间
10	0.85	0.15				
20	0.85	0.15				
100	0.85	0.15				
100	0	0.15				
100	0.5	0.15				
100	1	0.15				
100	0.85	0				
100	0.85	0.5				
100	0.85	1				

3. 设置种群规模为 100,交叉概率为 0.85,变异概率为 0.15,然后增加 1 种变异策略(如相邻两点互换变异、逆转变异或插入变异等)和 1 种个体选择概率分配策略(如按线性排序或者按非线性排序分配个体选择概率)用于求解同一 TSP 问题(如 10 个城市),把结果填入表 B.11。

表 B.11 不同的变异策略和个体选择概率分配策略的求解结果

变异策略	个体选择概率分配	最好适应度	最差适应度	平均适应度	平均运行时间
两点互换	按适应度比例分配				

四、实验报告要求

1. 画出遗传算法求解 TSP 问题的流程图。
2. 分析遗传算法求解不同规模的 TSP 问题的算法性能。
3. 对于同一个 TSP 问题,分析种群规模、交叉概率和变异概率对算法结果的影响。
4. 增加 1 种变异策略和 1 种个体选择概率分配策略,比较求解同一 TSP 问题时不同变异策略及不同个体选择分配策略对算法结果的影响。
5. 总结实验心得体会。

实验八 粒子群算法求函数最小值实验

一、实验目的

熟悉和掌握粒子群算法的原理、流程,并利用粒子群算法求解函数优化问题,理解求解流程并测试主要参数对实验结果的影响。

二、实验内容

用粒子群算法求解函数优化问题。

三、实验要求

1. 用粒子群求解下列函数的最小值。

测试函数:

$$f(x)=\sum_{i=1}^D\frac{x_i^2}{4\,000}-\prod_{i=1}^D\cos\frac{x_i}{\sqrt{i}}+1$$

其中, D 为变量维度, $x_i\in[-600,600],\forall i=1,2,\cdots,D$

- (1) 使用 Python 编写粒子群算法代码,并给出该适应度函数的定义方法。
- (2) 记录每一代种群的全局最优个体 g_best 的适应度值,使用 Matplotlib 可视化 g_best 适应度值随种群迭代的变化曲线,理解并分析种群收敛过程。
- (3) 设置不同的加速度常量,求得相应的最佳适应度、平均适应度,填入表 B. 12,比较并分析 PSO 社会模型、PSO 认知模型等对算法性能的影响。

表 B. 12 不同加速度的 PSO 运行结果

φ_1	φ_2	最佳适应度	平均适应度
2	0		
0	2		

2. 用粒子群局部版本(利用环形法,粒子的编号取粒子的邻域)求解同一函数的最小值问题。
- (1) 记录每一代种群的全局最优个体 g_best 的适应度值,使用 Matplotlib 可视化 g_best 适应度值随种群迭代的变化曲线。
 - (2) 比较分析粒子群全局版本和局部版本的差别。

四、实验报告要求

- 1. 画出粒子群算法的算法流程图。
- 2. 根据实验内容,给出相应结果以及结果分析。
- 3. 比较分析粒子群局部版本和全局版本的区别。

实验九 蚁群算法求最短路径实验

一、实验目的

熟悉和掌握蚁群算法的原理、流程和算法模型,并利用蚁群算法求解最短路径问题,理解求解流程并测试主要参数对结果的影响。

二、实验内容

编写基于蚁群算法求解 TSP 问题的程序,算出 20 个城市和 50 个城市之间的最短距离。

三、实验要求

- 1. 编写基于蚁群算法求解 TSP 问题的程序,给出 20 个城市的坐标信息,求解最短路径;将

输入数据换为 50 个城市的坐标,求解最短路径。

2. 对于同一个 TSP 问题,设置不同的参数(信息素启发式因子 α 、期望启发式因子 β 、信息素残留常数 ρ 、蚁群规模等),分析不同的参数对蚁群算法的影响。
3. 讨论三种不同的 $\Delta\tau_{xy}^k(t)$ 模型,分析哪种模型的效果最佳。

四、实验报告要求

1. 画出蚁群算法求解 TSP 问题的流程图。
2. 针对同一个 TSP 问题,分析不同的参数对于蚁群算法效果的影响。
3. 分析蚁群算法和遗传算法的区别与联系,优缺点。
4. 总结实验心得。

实验十 基于神经网络的模式识别实验

一、实验目的

理解 BP 神经网络和离散 Hopfield 神经网络的结构和原理,掌握反向传播学习算法对神经元的训练过程,了解反向传播公式。通过构建 BP 网络和离散 Hopfield 网络模式识别实例,熟悉前馈网络和反馈网络的原理及结构。

二、实验内容

Matlab 7. X 的神经网络工具箱:在 Matlab 7. X 的命令窗口输入 nntool,然后在键盘上输入 Enter 键,即可打开神经网络工具箱。

三、实验要求

1. 针对教材例 8. 1,设计一个 BP 网络结构模型(63-6-9),并以教材图 8. 5 为训练样本数据,图 8. 6 为测试数据。
- (1) 从 Matlab 工作空间导入(import)训练样本数据(inputdata, outputdata)和测试数据(testinputdata),然后新建一个神经网络(new network),选择参数如表 B. 13 所示,给出 BP 神经网络结构图。

表 B. 13 BP 网络结构模型的各项参数设置

network name(神经网络名称)	
network type(神经网络类型)	feed-forward backprop(前馈反向传播)
input ranges(输入信息范围)	来自训练样本的输入数据(inputdata)

续表

training function(训练函数)	TRAINGD(梯度下降 BP 算法)
performance function(性能函数)	MSE(均方误差)
number of layers(神经网络层数)	2
layer1(第 1 层)的 number of neurons(神经元个数)	6
layer1(第 1 层)的 transfer function(激活函数)	TANSIG(双曲正切 S 型函数)
layer2(第 2 层)的 number of neurons(神经元个数)	2
layer2(第 2 层)的 transfer function(激活函数)	LOGSIG(S 型函数)

(2) 输入训练样本数据(inputdata/outputdata),随机初始化连接权(initialize weights),给出 BP 神经网络训练成功后的误差变化曲线图,训练参数设置如表 B. 14 所示。

表 B. 14 BP 网络训练参数

训练次数(epochs)	1 000
训练时间(time)	Inf
训练目标(goal)	0
学习率(lr)	0.3
最大确认失败次数(max_fail)	5
最小性能梯度(min_grad)	1e-025
两次显示之间的训练步数(show)	25

(3) 选择不同的训练函数,例如 TRAINGDM(梯度下降动量 BP 算法)、TRAINLMM(levenberg-marquardt BP 训练函数),然后输入训练样本数据(inputdata/outputdata),训练参数设置如表 B. 14 所示,设置相同的初始连接权(revert weights),观察不同 BP 训练算法的学习效果,给出各训练算法下的误差变化曲线图。

(4) 在上述 3 个训练好的 BP 神经网络中,选择训练误差最小的一个网络,并给出训练后的连接权值和偏置,然后输入测试数据(testinputdata)进行仿真(simulation),并把训练和测试的结果都导出到工作空间,给出训练后的输出结果和输出误差,以及测试后的输出结果和输出误差。

(5) 针对 training function(训练函数)为 TRAINGD 的 BP 网络,设置不同的学习率(lr),如 0.01、0.1、0.5、1,观察 TRAINGD 训练算法的学习效果,给出各学习率下的误差变化曲线图。

2. 已知字符点阵为 2×2 模式,两组训练数据如图 B.3 所示。

$$a^{(1)} = [1 \quad 0 \quad 1 \quad 1]^L \quad (\text{大写字母 L})$$
$$a^{(2)} = [0 \quad 1 \quad 0 \quad 1]^l \quad (\text{小写字母 l})$$

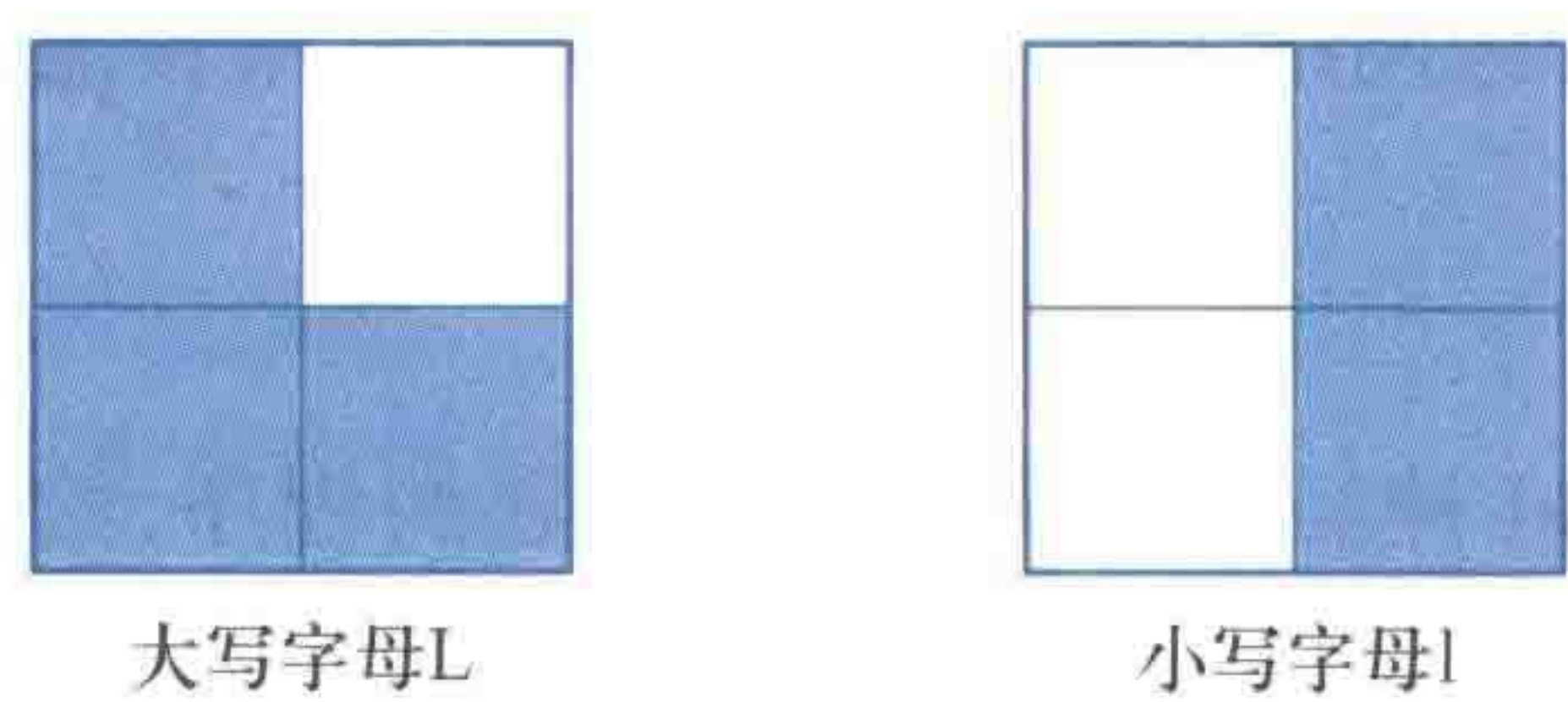


图 B.3 训练数据

设计一个能够存储这两个字符的离散 Hopfield 神经网络,要求:

- (1) 给出相应的离散 Hopfield 神经网络结构图;
- (2) 计算连接权值及阈值(阈值可设为 0);
- (3) 输入下列测试数据,如图 B.4 所示。

$$t^{(1)} = [1 \quad 1 \quad 0 \quad 1]^T$$
$$t^{(2)} = [0 \quad 0 \quad 1 \quad 1]^T$$

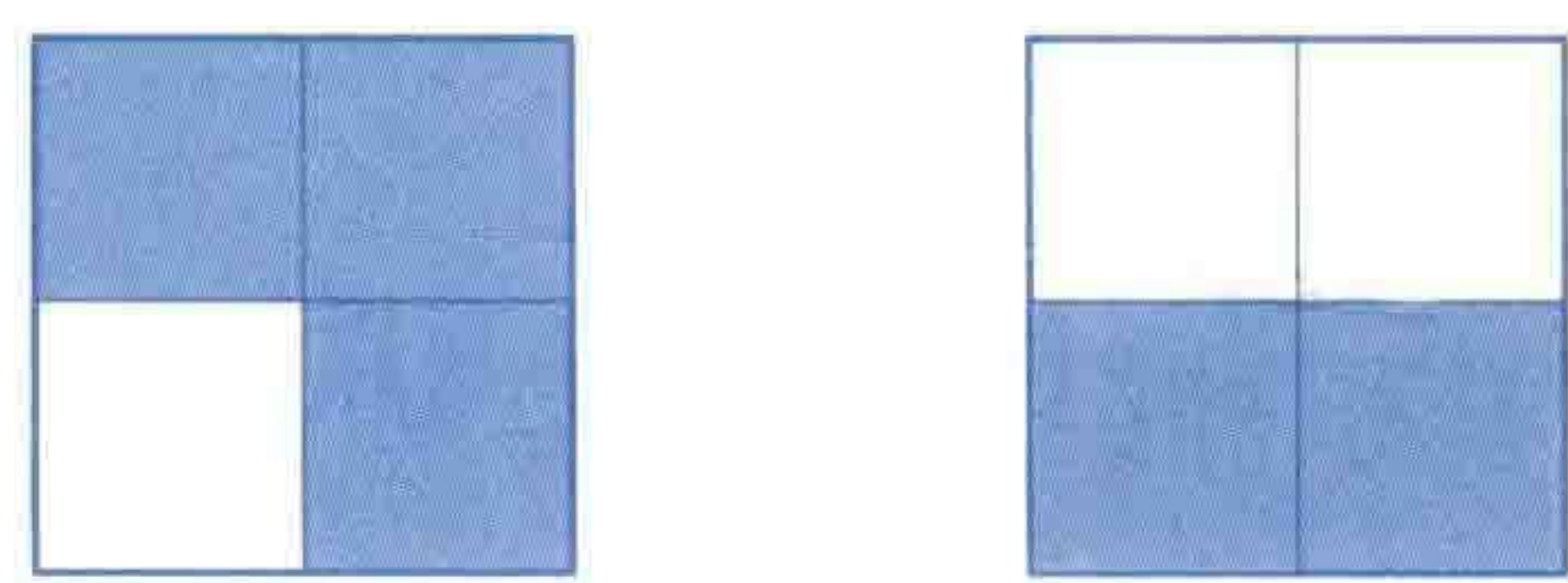


图 B.4 测试数据

给出网络最终输出的稳定状态。

四、实验报告要求

- 1. 按照实验内容,给出相应结果。
- 2. 分析比较采用梯度下降训练算法的 BP 网络学习率变化对训练结果的影响。
- 3. 分析比较 BP 网络和离散 Hopfield 网络在模式识别方面的异同点。
- 4. 总结实验心得体会。

实验十一 Hopfield 神经网络的优化计算实验

一、实验目的

掌握连续 Hopfield 神经网络的结构和运行机制,理解连续 Hopfield 神经网络用于优化计算

的基本原理,掌握连续 Hopfield 神经网络用于优化计算的一般步骤。

二、实验内容

编写基于连续 Hopfield 神经网络求解 TSP 问题的程序(编程语言不限,如 Python、VC++等)。

三、实验要求

1. 编写基于连续 Hopfield 神经网络求解 TSP 问题的程序,给出 15 个城市和 20 个城市的求解结果(包括最短路径和最佳路线),分析连续 Hopfield 神经网络求解不同规模 TSP 问题的算法性能。

2. 对于同一个 TSP 问题(例如 15 个城市的 TSP 问题),设置不同的网络参数,分析不同参数对算法结果的影响。

四、实验报告要求

1. 画出连续 Hopfield 神经网络求解 TSP 问题的流程图。
2. 根据实验内容,给出相应结果及分析。
3. 总结连续 Hopfield 神经网络和遗传算法用于 TSP 问题求解时的优缺点。
4. 总结实验心得体会。

实验十二 卷积神经网络分类 MNIST 数据集实验

一、实验目的

熟悉掌握卷积神经网络的定义,了解网络中卷积层、池化层等各层特点,并利用卷积神经网络对 MNIST 手写数字数据集进行分类。

MNIST 数据集来自美国国家标准与技术研究所。训练集由 250 个不同人手写的数字构成,其中 50%是高中学生,50%来自人口普查局的工作人员。

二、实验内容

编写卷积神经网络分类软件,编程语言不限,如 Python 等。以 MNIST 数据集为数据,实现对 MNIST 数据集分类的操作,其中 MNIST 数据集有 10 类,分别为手写数字 0~9。

三、实验要求

1. 用 MNIST 数据集训练编写的网络,要求记录每次迭代的损失值。
2. 改变卷积神经网络卷积层和池化层数,观察分类准确率。思考网络层数的多少对分类

准确率的影响。

3. 改变卷积神经网络卷积核大小,观察分类准确率。思考网络卷积核大小对分类准确率的影响。

四、实验报告要求

1. 按照实验要求,给出相应结果。
2. 分析卷积层和池化层数对网络的影响。
3. 分析卷积核大小对网络的影响。
4. 总结实验心得体会。

实验十三 胶囊网络分类 MNIST 数据集实验

一、实验目的

熟悉掌握胶囊网络的定义,了解网络中胶囊、动态路由等特点,并利用胶囊神经网络对 MNIST 数据集进行分类。

二、实验内容

编写胶囊网络分类软件,编程语言不限,如 Python 等。以 MNIST 数据集为数据,实现对 MNIST 数据集分类的操作,其中 MNIST 数据集有 10 类,分别为手写数字 0~9。

三、实验要求

1. 用 MNIST 数据集训练编写的网络,要求记录每次迭代的损失值。
2. 改变胶囊网络胶囊的参数,观察损失值和分类准确率。思考参数的改变对分类准确率的影响。
3. 去掉重构层以及重构损失,观察损失值和分类准确率。思考为什么要增加重构层来加快训练。

四、实验报告要求

1. 按照实验要求,给出相应结果。
2. 分析胶囊参数对网络的影响。
3. 分析重构层对网络的影响。
4. 总结实验心得体会。

实验十四 用生成对抗网络生成数字图像实验

一、实验目的

熟悉和掌握生成对抗网络的定义、生成器和判别器,并利用生成对抗网络生成数字图像。理解网络模型的对抗过程。

二、实验内容

编写生成对抗网络生成数字图像软件,编程语言不限,如 Python 等。以 MNIST 数据集为数据,用 GAN 实现手写数字 5 的图像生成。

三、实验要求

1. 用 MNIST 数据集训练编写的网络,要求记录每次迭代生成器和判别器的损失函数值。
2. 每隔固定代数生成一次图片,比较随代数增加图像的清晰程度。思考在一定程度后图像清晰度不再变化的原因。

四、实验报告要求

1. 按照实验要求,给出相应结果。
2. 分析 GAN 中生成器和判别器的关系。
3. 总结实验心得体会。

参 考 文 献

- [1] 王万良. 人工智能及其应用[M]. 4版. 北京:高等教育出版社,2020.
- [2] 王万良,吴启迪. 生产调度智能算法及其应用[M]. 北京:科学出版社,2007.
- [3] 袁南儿,王万良,苏宏业. 计算机新型控制策略及其应用[M]. 北京:清华大学出版社,1998.
- [4] 涂序彦. 人工智能及其应用[M]. 北京:电子工业出版社,1988.
- [5] 蔡自兴,徐光佑. 人工智能及其应用[M]. 3版. 北京:清华大学出版社,2003.
- [6] 王永庆. 人工智能原理与方法[M]. 西安:西安交通大学出版社,1998.
- [7] 王士同. 人工智能教程[M]. 北京:电子工业出版社,2001.
- [8] 王万良. 现代控制工程[M]. 2版. 北京:高等教育出版社,2021.
- [9] 李德毅,等. 人工智能导论[M]. 北京:中国科学技术出版社,2018.
- [10] 马少平,朱小燕. 人工智能[M]. 北京:清华大学出版社,2004.
- [11] 涂序彦,韩力群. 人工智能:回顾与展望[M]. 北京:科学出版社,2006.
- [12] 刘峡壁. 人工智能导论:方法与系统[M]. 北京:国防工业出版社,2008.
- [13] 李鸣华. 人工智能及其应用[M]. 北京:科学出版社,2008.
- [14] LUGER G F. Artificial Intelligence[M]. Fourth Edition. NewYork: Pearson Education Limited,2002.
- [15] 赵燕伟,张景玲,王万良. 物流配送的车辆路径优化方法[M]. 北京:科学出版社,2014.
- [16] 王海燕,等. 分批优化调度及差分进化算法[M]. 北京:科学出版社,2017.
- [17] [美]伊恩·古德费洛,[加]约书亚·本吉奥,[加]亚伦·库维尔. 深度学习[M]. 北京:人民邮电出版社,2017.
- [18] 张成文. 大模型导论[M]. 北京:人民邮电出版社,2024.
- [19] 徐俊刚. 深度学习[M]. 北京:机械工业出版社,2024.
- [20] BUCKLAND M. 游戏编程中的人工智能技术[M]. 吴祖增,沙鹰,译. 北京:清华大学出版社,2006.
- [21] FUNGE J D. 人工智能在计算机游戏和动画中的应用[M]. 班晓娟,艾迪明,译. 北京:清华大学出版社,2004.
- [22] FUNGE J D. 游戏人工智能——计算机游戏中的人工智能[M]. 李睿凡,郭燕慧,吴昕,译. 北京:北京邮电大学出版社,2007.

[23] BOURG D M, SEEMANN G. 游戏开发中的人工智能[M]. O'ReillyTaiwan 公司, 编译. 南京: 东南大学出版社, 2006.

[24] 郭燕慧. 游戏人工智能——计算机游戏中的人工智能[M]. 北京: 北京邮电大学出版社, 2007.



新形态教材网

网址：<https://abooks.hep.com.cn/65735>

使用方法详见书内新形态教材网使用说明

ISBN 978-7-04-065735-7



9 787040 657357 >

定价 55.00元